

فريق كتابة المقررات الجامعية

10

بنيان الحاسب - Computer architecture

1 مقرر الدكتور سامي

السلام عليكم يا أصدقاء

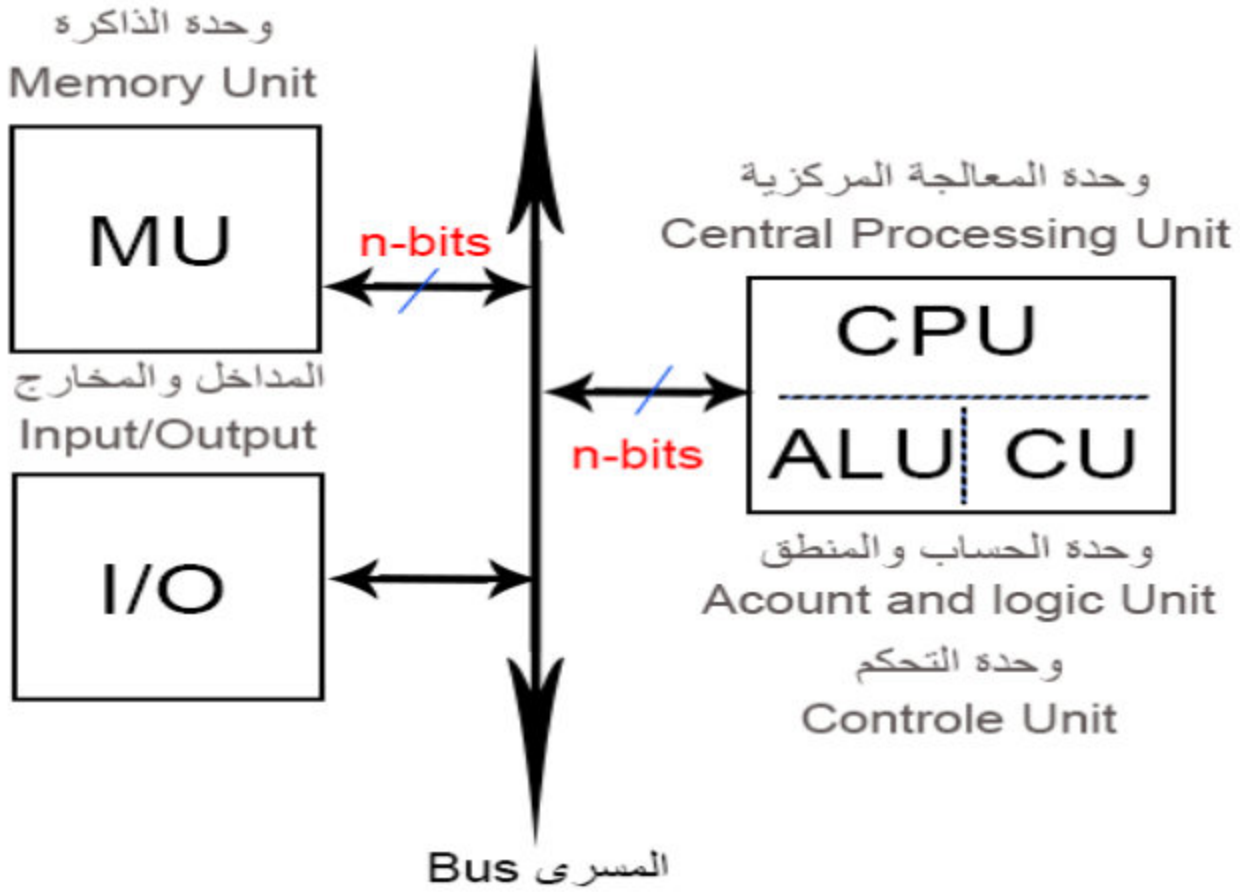
نقدم لكم الجزء الأول من مادة بنيان الحاسب للدكتور سامي عيسى

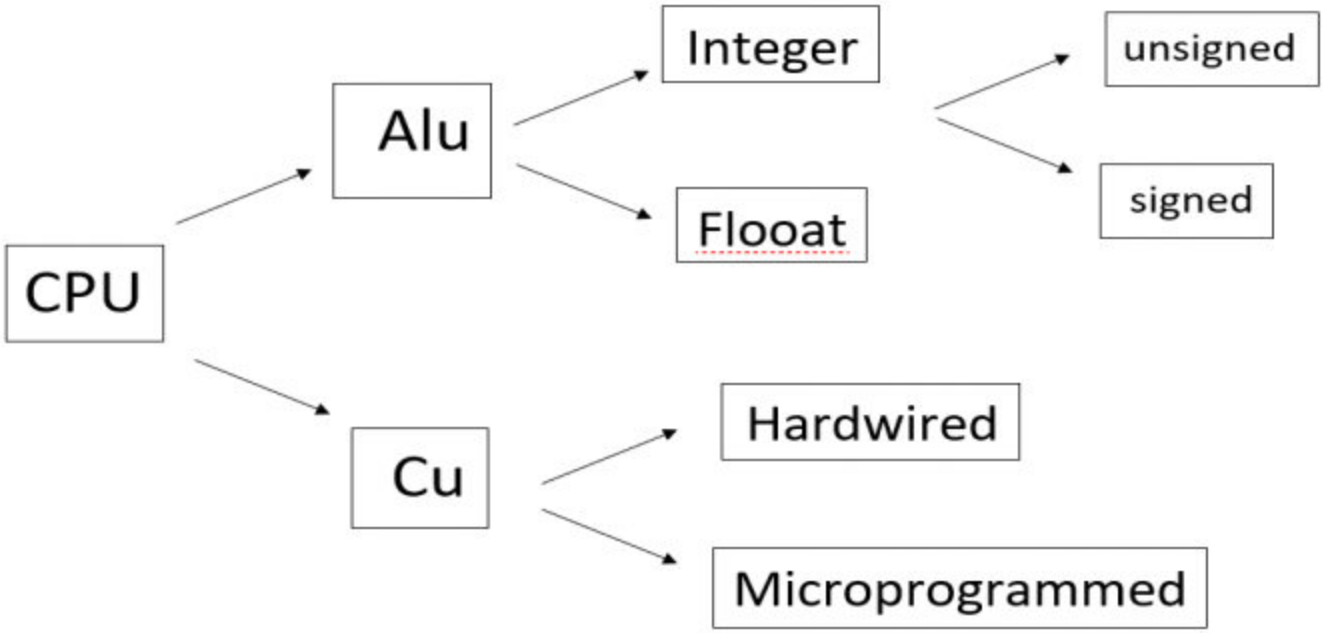
نتمنى لكم دوام التوفيق 😊

المحاضرات	رقم الصفحة
المحاضرة الأولى	1
المحاضرة الثانية	7
المحاضرة الثالثة	18
المحاضرة الرابعة	23
المحاضرة الخامسة	30
المحاضرة السادسة	37

المحاضرة الأولى

مقدمة: الأجزاء الرئيسية لمعمارية الحاسوب





بنيان الحاسوب وتنظيمه:

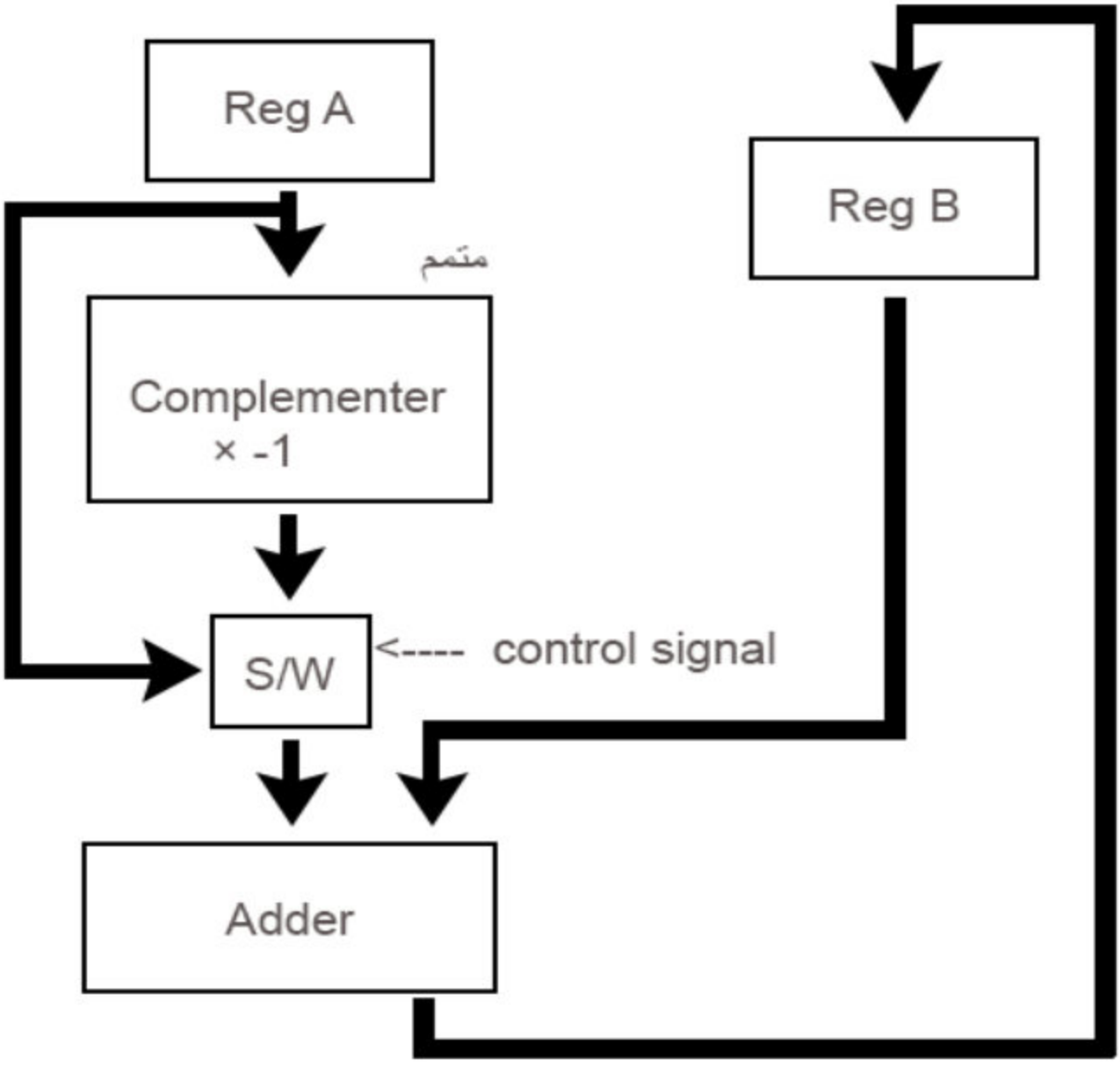
البنيان: ندرس البنى الموجودة ضمن الحاسوب من ذواكر ومعالج ...

التنظيم: آلية التفاعل بين الوحدات وآلية التنقل بينهم والبروتوكولات - تقنيات الذاكرة المستخدمة - إشارات التحكم - عدد البتات - تمثيل المعطيات.

سنبدأ بالدارات الموجودة ضمن ALU :

- دارة جمع وطرح الأعداد الصحيحة

المخطط الصندوقي للدارة :



- B <-- A ∓ B : العملية التي يقوم بها المخطط الصندوقي
- S/W : عبارة عن سويتش له دخلان
- A, -A : يمرر أحد المداخل وذلك عن طريق الإشارة التي تأتي من وحدة التحكم cu .
- Adder : دارة الجامع
- Reg A, RegB : العددان المراد جمعهما أو طرحهما
- Complementer : المتمم ونميز ثلاث أنواع : لنأخذ مثال العدد 5

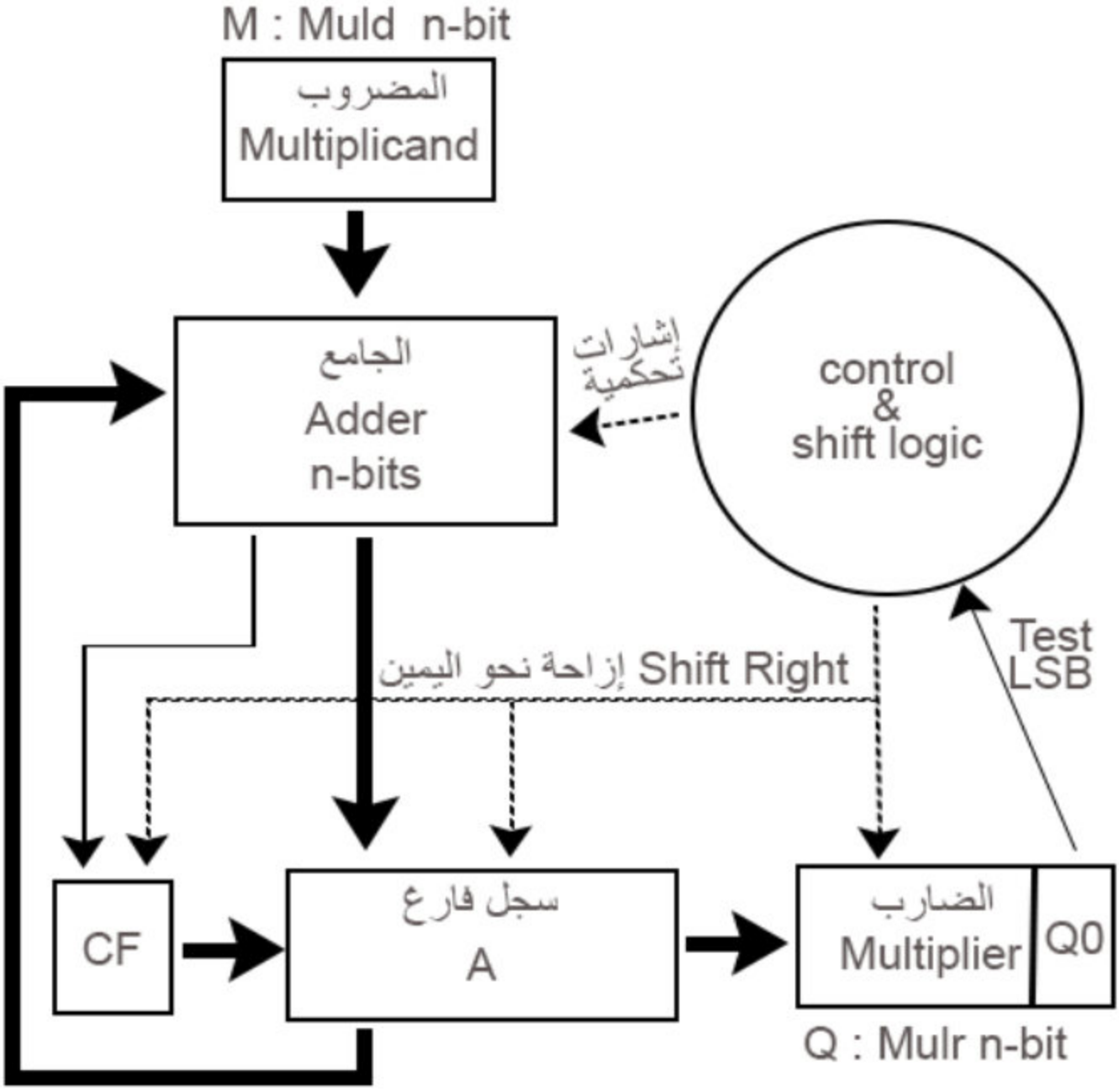
-5	5	الصيغة
1101 نضيف 1 من اليسار للعدد السالب	0101	Signed Magnitude
1010 نبدل كل 0 ب 1 ونبدل كل 1 ب 0	0101	1's Complement
1011 نبدل كل 0 ب 1 ونبدل كل 1 ب 0 نضيف 1	0101	2's Complement

ملاحظة:

سنعتمد في دراستنا كلها على المتمم الثنائي 2's complement عدا دارة القسمة للأعداد المؤشرة فإننا سنستخدم signed magnitude أي طويلة وإشارة .

- دارة ضرب الأعداد الصحيحة غير المؤشرة:

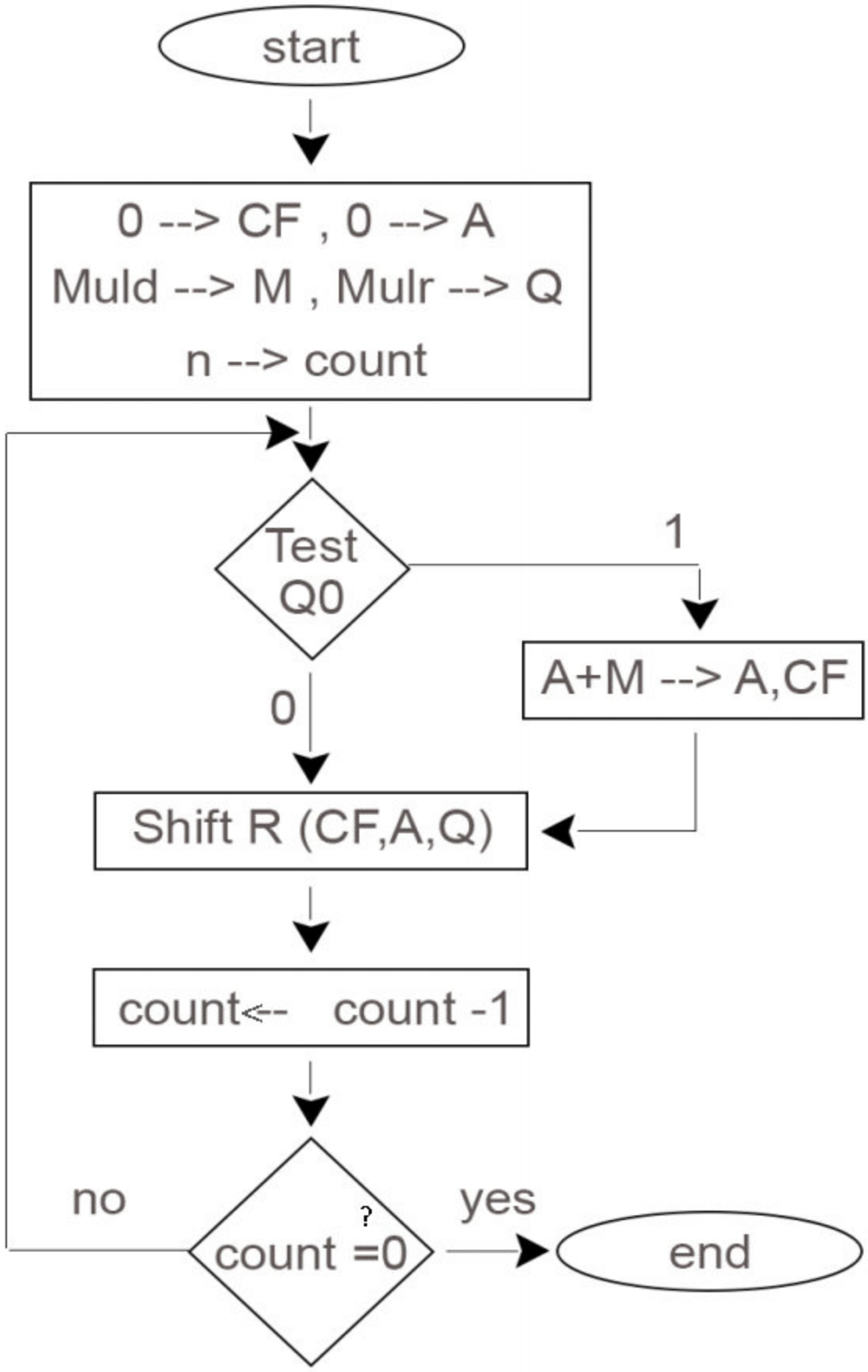
المخطط الصندوقي/العتادي :



CF : cary المنقول

ربطنا كل من A و Q لاستغلال الموارد دون الحاجة لسجل إضافي

الخوارزمية:


start
0 --> CF , 0 --> A
Muld --> M , Mulr --> Q
n --> count
Test Q0
1
A+M --> A,CF
0
Shift R (CF,A,Q)
count<-- count -1
count =0 ?
no
yes
end

التتبع : لنأخذ مثال ضرب 4 * 5 :

```
  0100
  0101   ×
------
  0100
 0000
0100
0000
-------
0010100
```

شرح العملية :

أولا : في عملية التعريف أو التهيئة initialization :

- نضع العدد الأول وهو المضروب 0100 (mulr) في السجل Q
- نضع العدد الثاني وهو الضارب 0101 (muld) في السجل M
- نضع 0 في كل من A و CF
- نضع عدد البتات في count

ثانيا : في الخطوة الأولى نفحص الخانة الأولى من العدد المضروب والذي في مثالنا هو 0100

- فإن كان 1 نقوم بجمع كل من السجل A و السجل M ونضع الناتج في A ثم نقوم بعلية إزاحة نحو اليمين Shift Right (Sh.R) .
- وإن كان 0 نقوم بالإزاحة نحو اليمين فقط .

نكرر العملية السابقة بعدد البتات وفي مثالنا 4

CF	A	Q	M ثابت	comment
0	0000	0100	0101	Initilization
0 →	0000 →	0010	0101	Sh.R
0 →	0000 →	0001	0101	Sh.R
0	0101	0001	0101	A+M -- >M
0 →	0010 →	1000	0101	Sh.R
0 →	0001 →	0100	0101	Sh.R

ناتج عملية الضرب = 20

ملاحظة :

قد يرد السؤال عن المخطط الصندوقي أوالخوارزمية أو التتبع.

المحاضرة الثانية

قانون تمثيل الأعداد غير المؤشرة : $0 \rightarrow 2^n - 1$

مثال : $0 \rightarrow 2^8 - 1$: n = 8 bit ---- >

0 ---- > 255

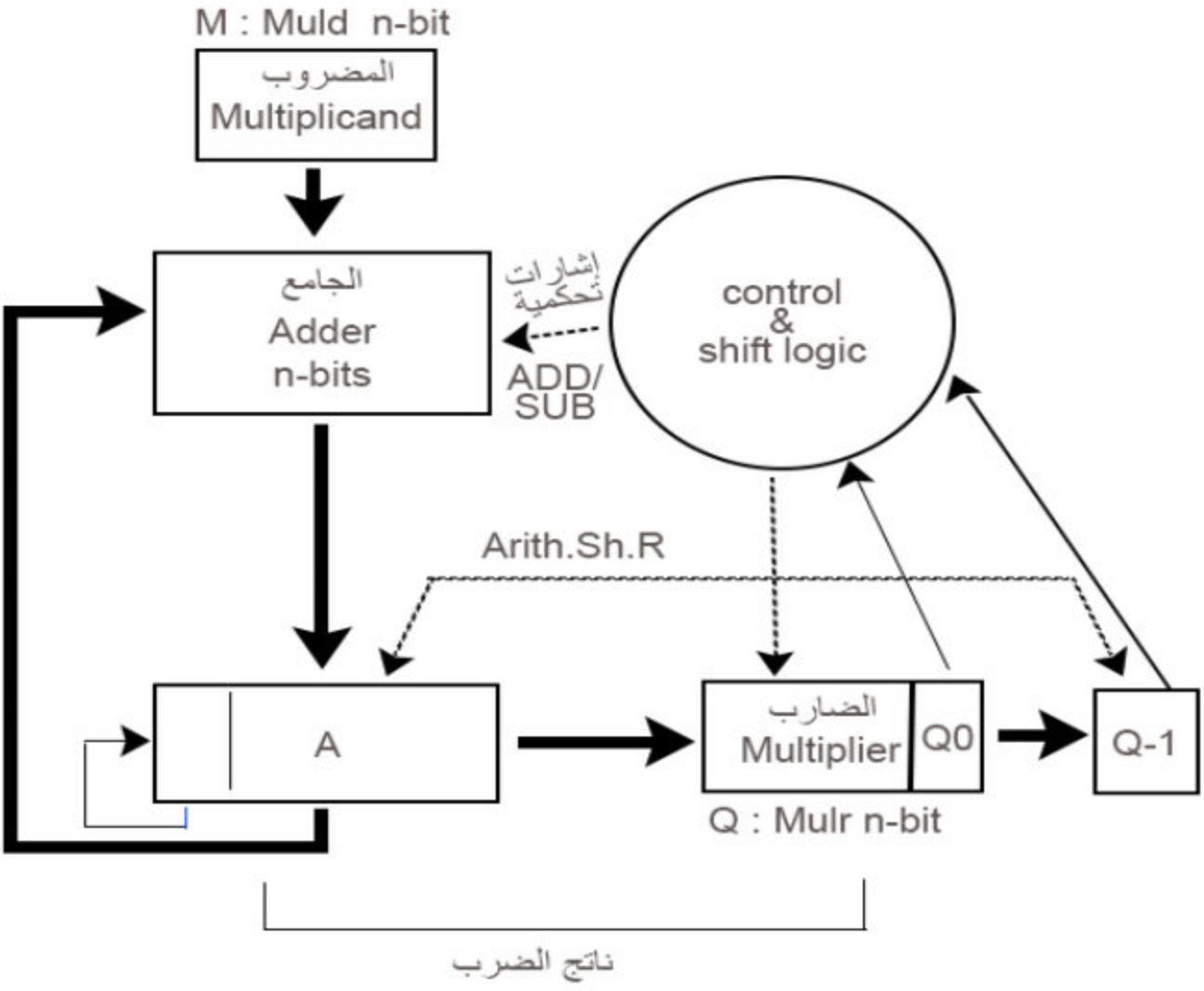
قانون تمثيل الأعداد المؤشرة : $-2^{n-1} \rightarrow +2^{n-1} - 1$

مثال : $-2^{7-1} \rightarrow +2^{7-1} - 1$ n = 8 bit ---- >

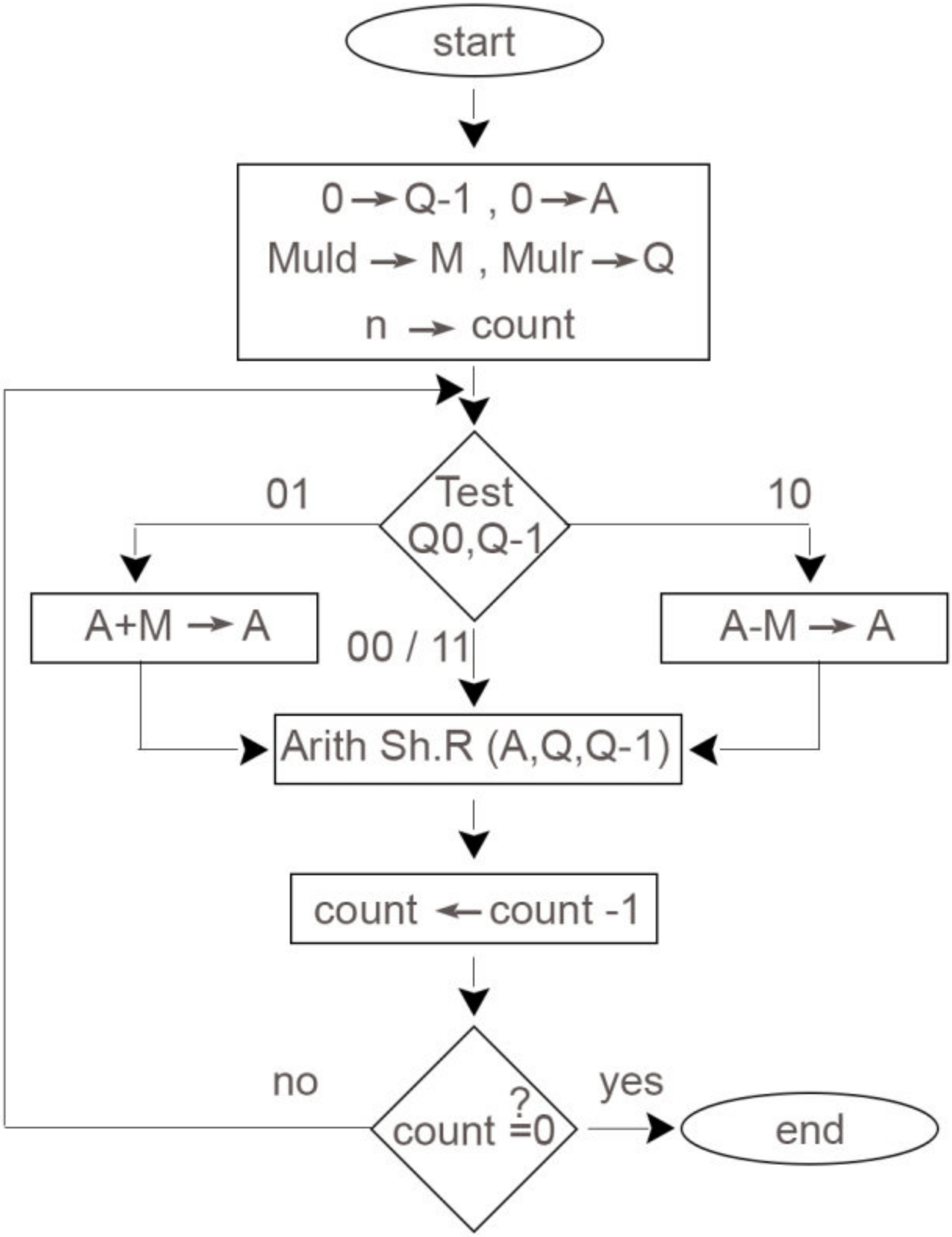
-128 ------ > 127

- دارة الضرب المؤشر (بووث) multiplication signed :

المخطط الصندوقي :



الخوارزمية : خوارزمية بووث:



التتبع : لنأخذ مثال (5-) * (3) :

A	Q	Q-1	M	comment
0000	0011	0	1011	init
0101	0011	0	1011	A-M → A
0010	1001	1	1011	Arith Sh.R
0001	0100	1	1011	Arith Sh.R
1100	0100	1	1011	A+M → A
1110	0010	0	1011	Arith Sh.R
1111	0001	0	1011	Arith Sh.R

ناتج الضرب

الناتج 11110001 هو 15- حيث أن

11110001 المتمم الثنائي للعدد 15

شرح العملية :

أولا : في عملية التهيئة : نضع الضارب ب Q ونضع المضروب ب M أو العكس لأن الضرب عملية تبديلية

ونضع 0 بعدد البتات ب A ونضع 0 ب Q-1 ونضع عدد البتات n في count في مثالنا 4

ثانيا : نقوم بعملية فحص لكل من Q0 و Q-1 :

_ فإن كان 01 نقوم بجمع كل من A و M ونضع الناتج في A ومن ثم نقوم بالإزاحة نحو اليمين كل من A و Q و Q-1

_ وإن كان 10 نقوم بطرح M من A أي (A - M) ومن ثم نضع الناتج في A ومن ثم نقوم بالإزاحة نحو اليمين كل من A و Q و Q-1

_ وإن كان 11 أو 00 نقوم بالإزاحة نحو اليمين كل من A و Q و Q-1

في كل مرة من العملية السابقة ينقص ال count مقدار1

تتكرر العملية السابقة حتى يصبح ال count = 0

سؤال امتحاني :

ما هو أكبر ناتج وأصغر ناتج يمكن الحصول عليه من ضرب عددين مكونين من 8 بت . ناقش الاحتمالات سواء الأعداد مؤشرة أم غير مؤشرة

الحل : العددين مكونيين من 8 بت

أولا : الأعداد غير المؤشرة :

بالعودة إلى قانون تمثيل الأعداد غير المؤشرة نجد أن :

أصغر ناتج : العدد الأول يمثل ب : $0 \rightarrow 2^n - 1$ وأصغر احتمال هو : 0 والعدد الثاني كذلك، فتصبح العملية هي : 0 * 0 = 0

أكبر ناتج :

العدد الأول يمثل ب : $0 \rightarrow 2^8 - 1$ وأكبر احتمال هو :$255 = 2^8 - 1$

والعدد الثاني كذلك، فتصبح العملية هي : 255 * 255 = 65025

ثانيا : الأعداد المؤشرة :

بالعودة إلى قانون تمثيل الأعداد المؤشرة نجد أن :

أصغر ناتج :

العدد الأول يمثل ب : $-2^7 \rightarrow +2^7 - 1$ وأصغر احتمال هو -2^7 = -128

العدد الثاني كذلك، وبما أنا نريد أصغر ناتج لعملية الضرب نأخذ

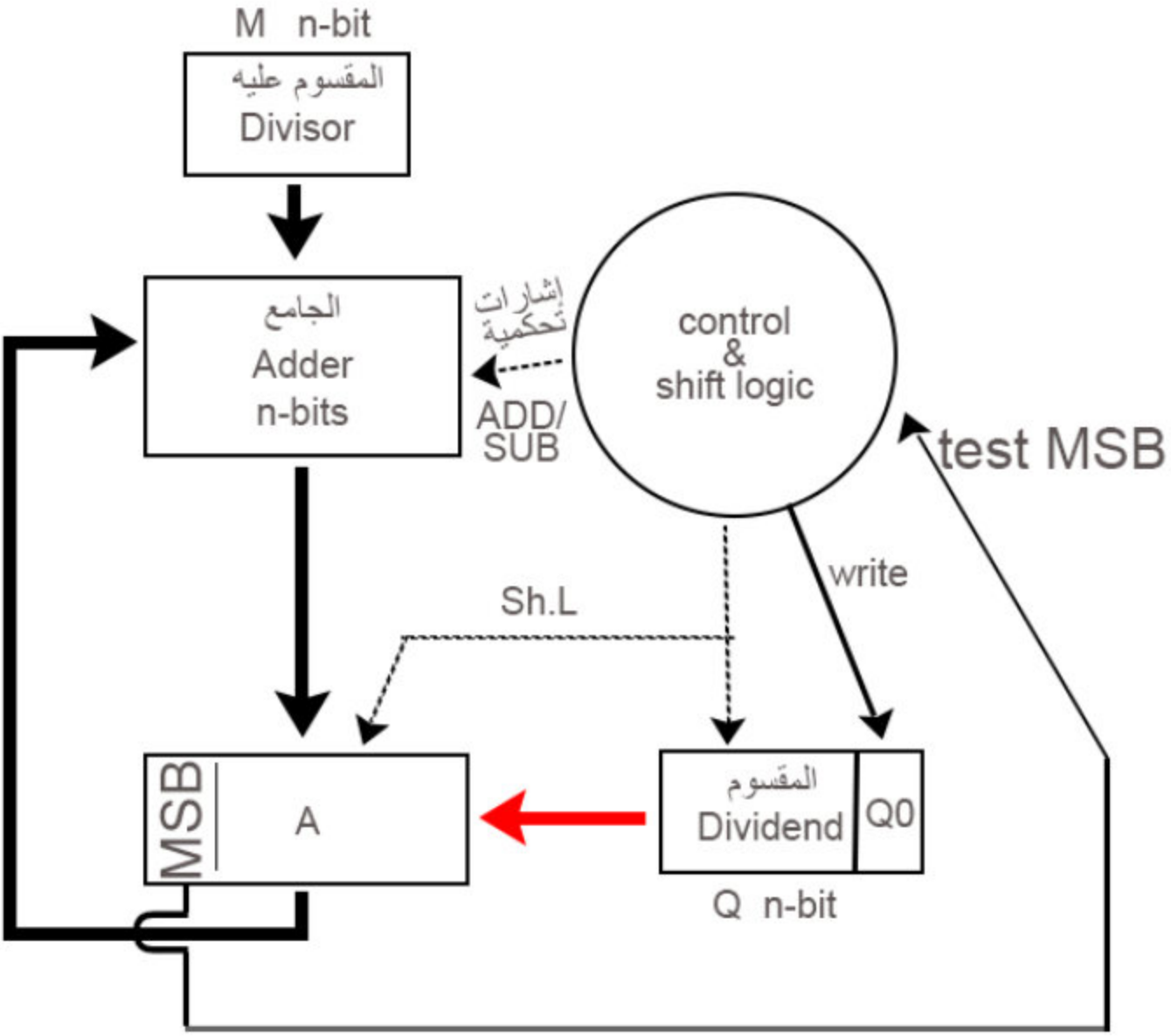
أصغر قيمة ونضربها مع أكبر قيمة فيصبح لدينا 128- * 127 = 16256-

أكبر ناتج : 128- * 128- = 16384

- دارة القسمة غير المؤشرة :

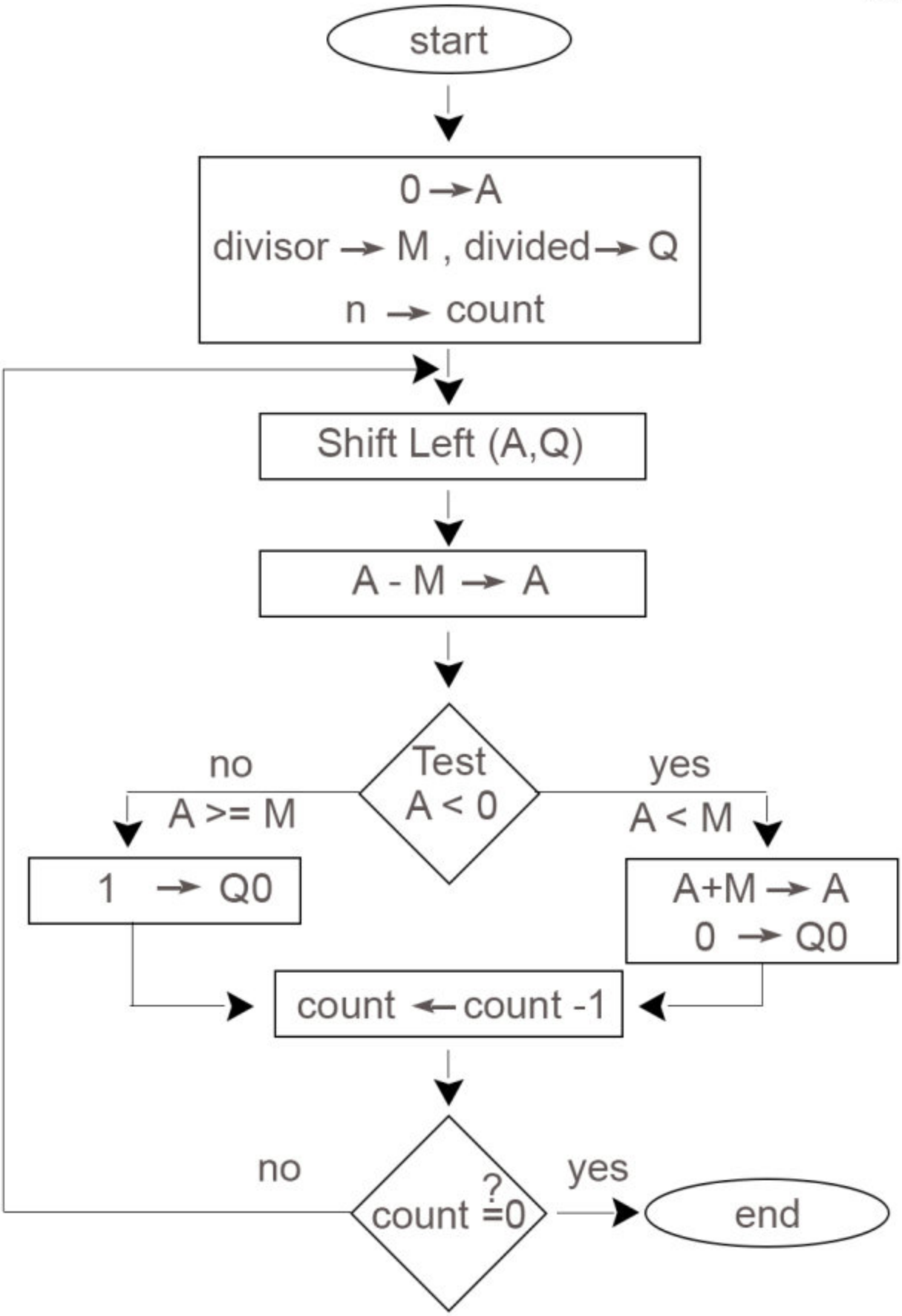
المخطط الصندوقي :





يكون الناتج في Q ويكون الباقي في A

الخوارزمية:


start
0 → A
divisor → M , divided → Q
n → count
Shift Left (A,Q)
A - M → A
Test
A < 0
no
A >= M
yes
A < M
1 → Q0
A+M → A
0 → Q0
count ← count -1
count =0 ?
no
yes
end

التتبع لنأخذ مثال العملية : 15/3:

A	Q	M	comment
0000	1111	0011	init
0001 ←	1110	0011	Sh.L (A,Q)
1110	1110	0011	A - M → A
0001	1110	0011	0 → Q0 A + M → A
0011 ←	1100	0011	Sh.L (A,Q)
0000	1100	0011	A - M → A
0000	1101	0011	1 → Q0
0001 ←	1010	0011	Sh.L (A,Q)
1110	1010	0011	A - M → A
0001	1010	0011	A <? 0 → yes 0 → Q0 A + M → A
0011 ←	0100	0011	Sh.L (A,Q)
0000	0100	0011	A - M → A
0000	0101	0011	A <? 0 → no 1 → Q0

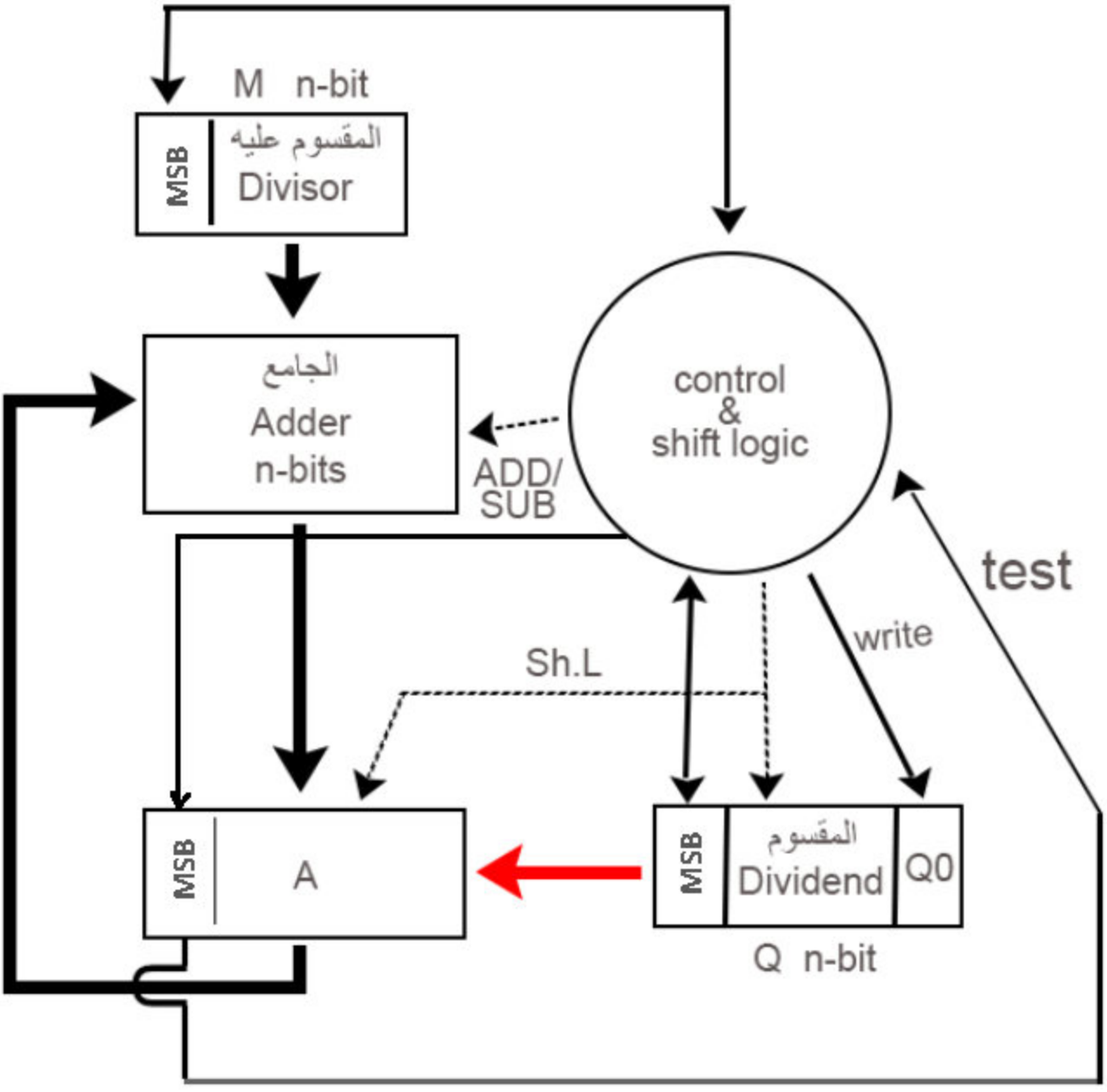
الباقي

ناتج القسمة

ن دارة القسمة المؤشرة :

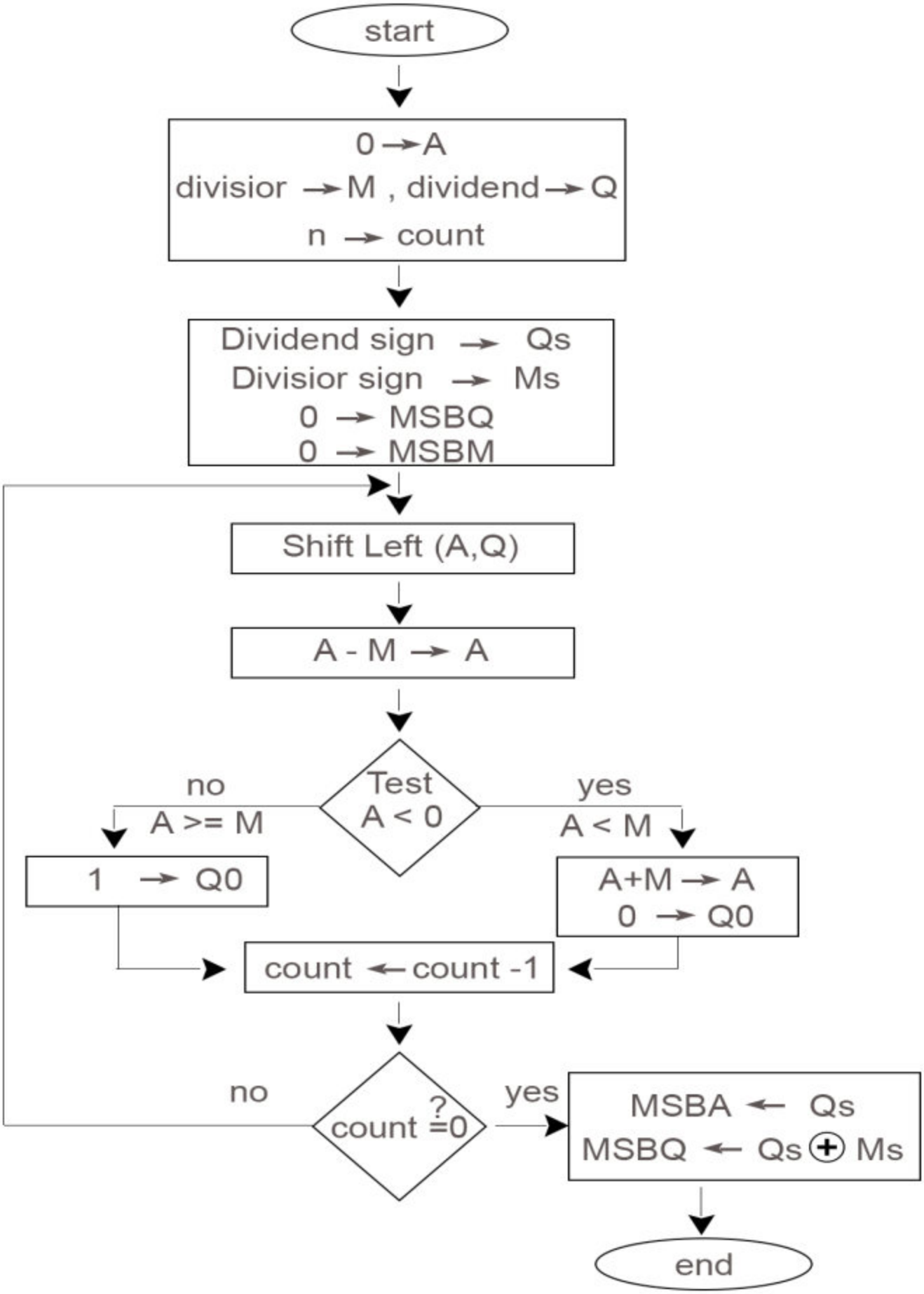
تذكرة :
في هذه الدارة نستخدم طريقة signed magnitude طويلة وإشارة

لمخطط الصندوقي :



ويكون ناتج القسمة في Q ويكون الباقي في A

الخوارزمية:


start
0 → A
divisior → M , dividend → Q
n → count
Dividend sign → Qs
Divisior sign → Ms
0 → MSBQ
0 → MSBM
Shift Left (A,Q)
A - M → A
Test
A < 0
no
A >= M
yes
A < M
1 → Q0
A+M → A
0 → Q0
count ← count -1
no
count =0 ?
yes
MSBA ← Qs
MSBQ ← Qs ⊕ Ms
end

التتبع : لنأخذ مثال 7/4-:

A	Q	M	comment
0000	1111	0100	Init
0000	0111	0100	Dividend s → Qs Divisior s → Ms 0 → MSBQ 0 → MSBM
0000	1110	0100	Sh.L (A,Q)
1100	1110	0100	A-M → A
0000	1110	0100	A+M → A 0 → Q0
0001	1100	0100	Sh.L (A,Q)
1011	1100	0100	A-M → A
0001	1100	0100	A+M → A 0 → Q0
0011	1000	0100	Sh.L (A,Q)
1001	1000	0100	A-M → A
0011	1000	0100	A-M → A 1 → Q0
0111	0000	0100	Sh.L (A,Q)
0011	0000	0100	A-M → A
0011	0001	0100	A+M → A 0 → Q0
1011	1001		Qs → MSBQ QS xor MS → MSBA

باقي القسمة

ناتج القسمة

ملاحظة:

تكون إشارة كل من الباقي والناتج على الشكل التالي :

إشارة الباقي	إشارة الناتج	العملية
+	+	+/+
+	-	+/-
-	-	-/+
-	+	-/-

المحاضرة الثالثة

أنماط تمثيل الأعداد الحقيقية Flooting Poing Representation

المعيار الأول ذو الدقة الأحادية Single precision 32 bit

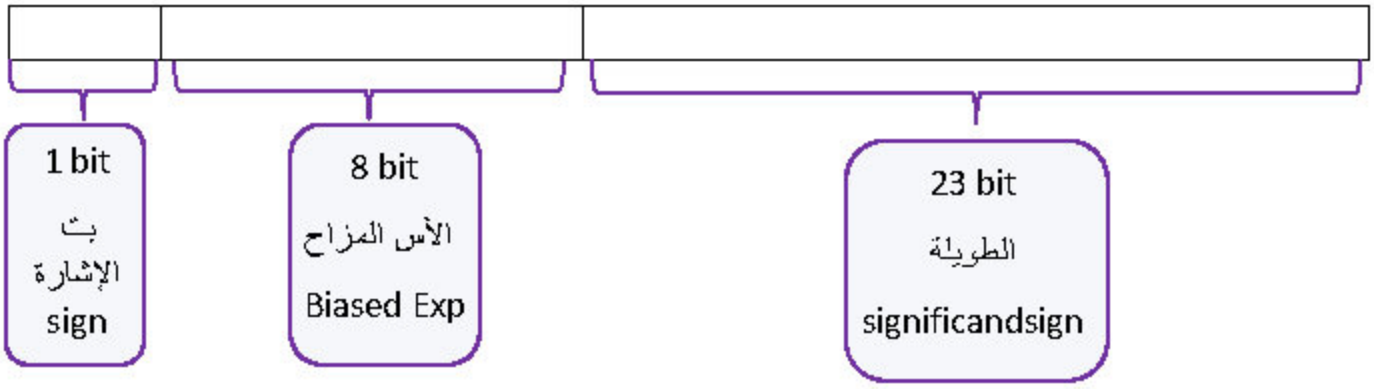
المعيار ذو الدقة المضاعفة Double precision 64 bit

لتمثيل الأعداد الحقيقية في الحاسب نستخدم المعيار IEEE754 وهو معيار تقني لعمليات الفاصلة العائمة.

وهذا المعيار يقوم بتعريف الصيغ الحسابية، مجموعة من بيانات الفاصلة العائمة الثنائية والعشرية مكونة من أرقام محددة. كما تشمل الأصفار ذات الإشارة (+0 و -0) واللانهايات بالإضافة إلى قيم غير رقمية تسمى NaN

نحن نستخدم (32 bit) داخل الحاسب العادي.

كيفية تمثيل العدد الحقيقي الى 32 bit



Sign bit : يعبر عن بت الإشارة فإذا كان 0 فالعدد موجب وإذا كان 1 فالعدد سالب

أي عدد حقيقي يكتب بالشكل التالي : $\mp M \times B^{\mp EXP}$

M : طويلة العدد

B : أساس نظام العد المستخدم

EXP : الأس الصحيح

نأخذ اصطلاحا أن 1 = M دوما وذلك في المعيار IEEE754 أي أن طويلة العدد يجب أن تكون 1 وعليه يكون الأس المزاج

ملاحظة :

في حال عدم ذكر نوع المعيار نأخذ المعيار الأحادي 32bit

قاعدة : في معيار 32 bit يكون الأس المزاج = الأس الحقيقي + ($2^{n-1}-1$) حيث n عدد بتات حقل الأس المزاج

مثال : n = 8 bit يكون الأس المزاج = الأس الحقيقي + 127

والفائدة من العملية السابقة لكي يتمثل الأس بعدد موجب دوما حيث نقوم بجمع له أكبر قيمة سالبة يمكن أن يتمثل بها حسب قاعدة تمثيل الأعداد في المحاضرة السابقة.

مثال : تمثيل العدد 22.625- في الحاسب :

- نحول العدد إلى عدد بنظام العد الثنائي مع مراعاة الإشارة يكون : 10110.101 -
- أضع العدد بشكل Normolization أوما يسمى التقييس أي يكون لدي على يسار الفاصلة خانة واحدة فقط وقيمتها (1) لذلك سنقوم بإزاحة الفاصلة أربع خطوات : $1.0110101 *2^{+4}$ -
- أخزن العدد الذي يكون على يمين الفاصلة في مكان 23 بت وأكمله باقي الخانات بأصفار
- أضع إشارة العدد في خانة الإشارة وبما أن العدد سالب سوف أضع (1)
- يبقى Biased Exponent فأضع الأس المزاج حسب القاعدة السابقة وبعد تحويله للنظام الثنائي وأكمل باقي الخانات بأصفار- في مثالنا : الأس الحقيقي 4 فيكون الأس المزاج 4 + 127 = 131 نحولها للنظام الثنائي فيكون : 10000011

يمثل العدد كالتالي:

0	01101010	1

التمثيل ضمن الذاكرة يكون على شكل :

00
00
B5
C1

حيث يتم التمثيل ضمن الذاكرة بنظام العد السداسي عشر – حيث حولنا العدد السابق كله إلى نظام سداسي عشر

والتحويل السداسي عشر للعدد السابق : 1100 0001 1011 0101 0000 0000 0000 0000

هو: C 1 B 5 0 0 0 0

ملاحظة:

غالبا ما يأتي بالامتحان على يمين الفاصلة : 5 و 625 و 25

سؤال امتحاني : التمثيل على خط الأعداد

1 - تمثيل ∞+ في الفاصلة العائمة

0000 00000	1111 1111	0

23 بت 8 بت 1 بت

حيث هو : $1.0 * 2^{255}$ و بالأس الحقيقي يكون $1.0 * 2^{128}$

2 - تمثيل ∞- في الفاصلة العائمة

0000 00000	1111 1111	1

حيث هو : $-1.0 * 2^{255}$ و بالأس الحقيقي يكون $-1.0 * 2^{128}$

3 - تمثيل 0+ في الفاصلة العائمة

0000 00000	0000 0000	0

حيث هو : $1.0 * 2^{0}$ و بالأس الحقيقي يكون $1.0 * 2^{-127}$

4 - تمثيل 0- في الفاصلة العائمة

1	0000 0000	00000 0000

حيث هو : $1.0 * 2^{0}$ -1.0 و بالأس الحقيقي يكون $-1.0 * 2^{-127}$

5 - تمثيل أكبر عدد في الفاصلة العائمة

0	1111 1111	111 1111

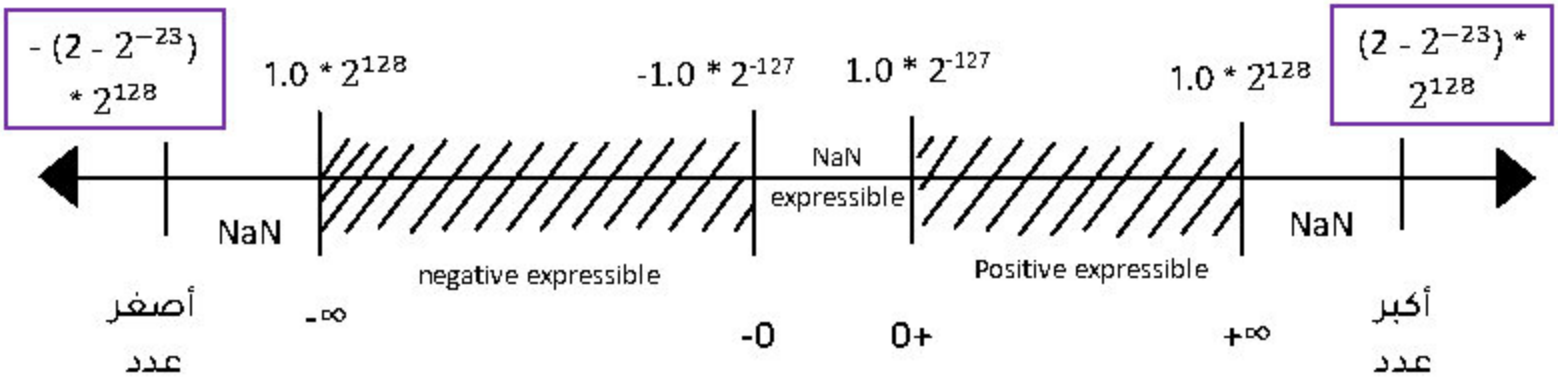
حيث هو : $1.11...1 * 2^{255}$ وبالأس الحقيقي يكون $1.11....1 * 2^{128}$ ويساوي تقريبا : $2 * 2^{128}$
ويساوي : $(2 - 2^{-23}) * 2^{128}$

6 - تمثيل أصغر عدد في الفاصلة العائمة

1	1111 1111	1111 1111

حيث هو : $-1.11...1 * 2^{255}$ وبالأس الحقيقي يكون $-1.1....1 * 2^{128}$

عدد احتمالات NaN هي $2^{24} - 2$



سؤال امتحاني :

لدينا : x = 3F00 0000 و y = C000 0000 أوجد

1) X * y

2) x / y

الحل: نحول كل عدد إلى نظام ثنائي ونمثله بالفاصلة العائمة

x = 0011 1111 0000 0000 0000 0000 0000 0000 = $1.0 * 2^{126}$ موجب

0	01111110	000 000

$-1.0 * 2^{128}$ = 0000 0000 0000 0000 0000 0000 0000 0000 1100 = y سالب

1	10000000	000 000

نقوم بإرجاع الأس الحقيقي لكلا العددين x و y أي نطرح من الأس المزاح مقدار 127 فينتج :

$y = -1.0 * 2^{1}$, $x = 1.0 * 2^{-1}$

1) x * Y نقوم بضرب العددين وجمع الأس

$X * y = 1.0 * 2^{-1} * (-1.0 * 2^{1}) = -1.0 * 2^{0}$

وعند تمثيله بالفاصلة العائمة نضيف للأس 127 حسب قاعدة الأس المزاح : $-1.0 * 2^{127}$

نمثل الناتج بالفاصلة العائمة

1	0111 1111	0000 0

وأخيرا نحول الناتج إلى نظام سداسي عشر فينتج : BF 80 00 00

2) x / y نقوم بقسمة العددين ونطرح الأس

$x / y = 1.0 * 2^{-1} / (-1.0 * 2^{1}) = -1.0 * 2^{-2}$

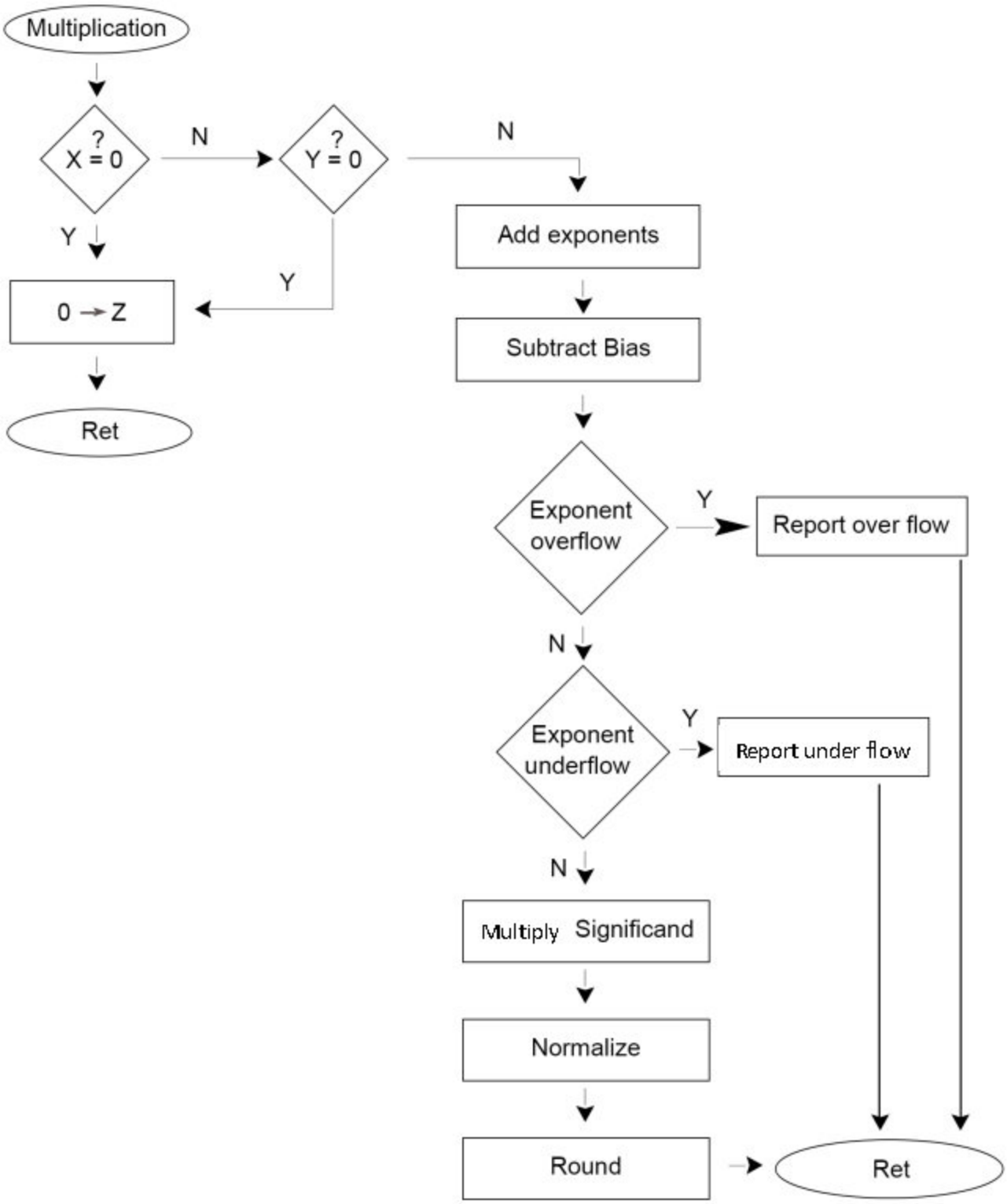
وعند تمثيله بالفاصلة العائمة نضيف للأس 127 حسب قاعدة الأس المزاح : $-1.0 * 2^{125}$

1	0111 1101	00 0

وأخيرا نحول الناتج إلى نظام سداسي عشر فينتج : BE 80 00 00

المحاضرة الرابعة

خوارزمية الضرب للأعداد العائمة x * y ⟵ z



شرح المخطط :

- التحقق من أن العددين لا يساويان الصفر وفي حال كان أحدهما يساوي الصفر فالناتج صفر
- نقوم بجمع الأسس
- طرح الإزاحة : كل من العددين فيه Bias وعند جمع الأسس أصبح ال Bias مجموع مرتين وبتمثيل 32bit ال Bias هو 127 ($2^{n-1} - 1$) ، وهذا يعني أنه أصبح لدينا إزاحة مضاعفة لذلك طرحنا
- التحقق من وجود over flow أو under flow لأن عند جمع الأسس من الممكن أن يكون الناتج خارج المجال

--> over flow أو بحدود الصفر

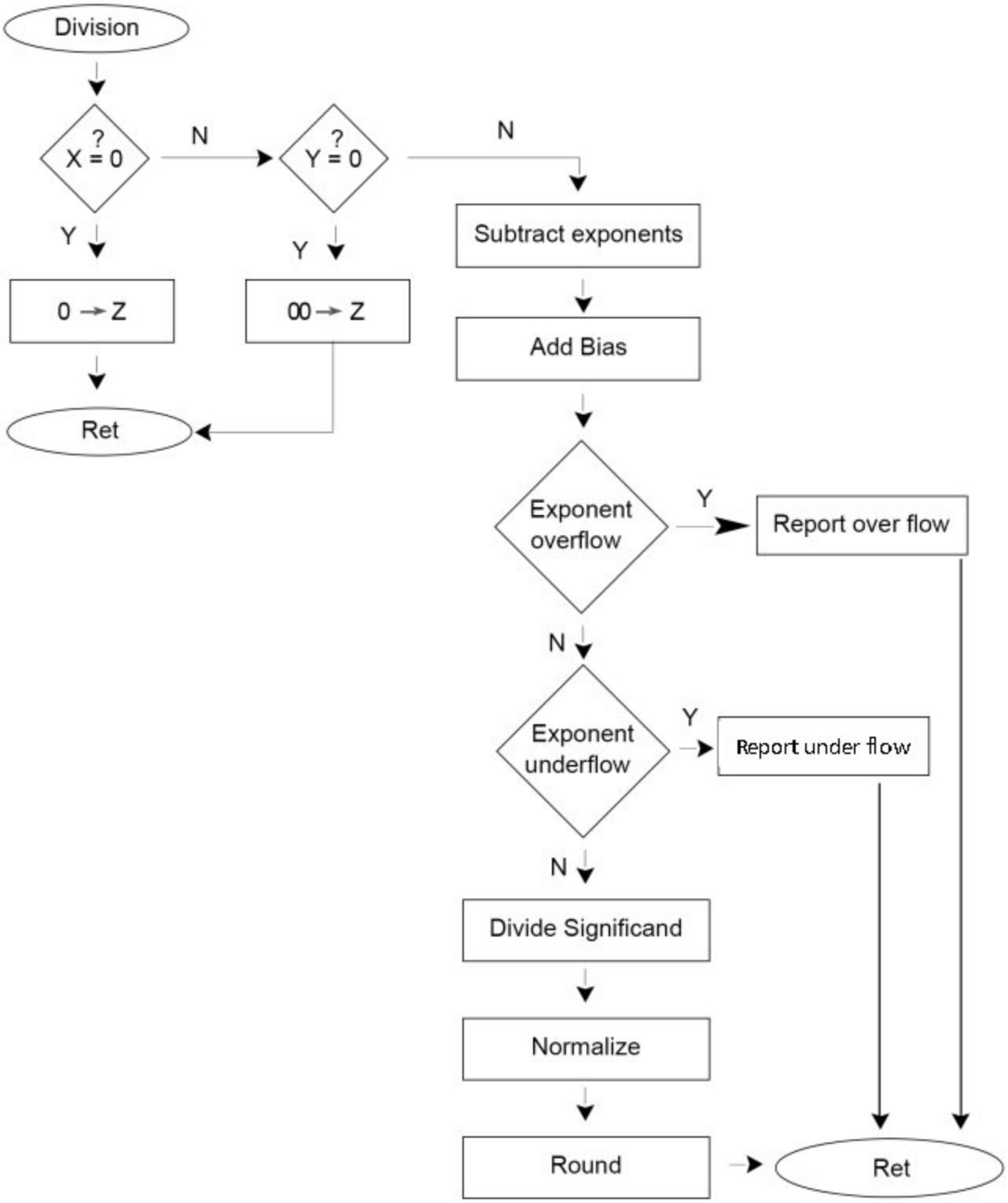
--> under flow وهي حالات غير قابلة للتمثيل

- في حال كان لدينا over أو under --> إظهار للمستخدم Report فإن الجواب غير قابل للتمثيل
- ضرب الطويلة
- التقييس (الصيغة العلمية) : خانة وحدة على يسار الفاصلة قيمتها 1
- التقريب.

ملاحظة :

سجل الأعلام : يعطي مؤشرات بعد القيام بالعمليات الحسابية مثل - over flow cary ....

خوارزمية القسمة للأعداج العائمة : X / Y ——> Z



وحدة التحكم : Cu

السجلات داخل المعالج :

User - Visiable.Reg : سجلات لأغراض عامة - معطيات - عنونة - شروط وأعلام

مهمتها : تسريع تنفيذ البرامج المكتوبة بلغة التجميع - مرئية بالنسبة للمستخدم

Control and Status .Reg : PC - IR - MDR/MBR - MAR

غير مرئية للمستخدم والمبرمج (سجلات موجودة ولكن غير مرئية ولا يوجد وصول للمبرمج الوصول فقط لنظام التشغيل)

مهمتها : التحكم في عمل وتشغيل المعالج

- سجل عداد البرنامج : Program Counter "PC" : يحفظ عنوان التعليمة التالية في التنفيذ
- Instruction Reg "IR" : سجل التعليمة أو مستقر التعليمة : نقوم بجلب التعليمات من الذاكرة ونضعها في IR تمهيدا لتنفيذها
- MDR و MAR : سجلات التعامل مع الذاكرة

Memory Address Register _ Memory Data Register

أي تعليمة نحولها لتعليمة ميكروية

إذا أردنا وضع تعليمة ضمن IR

صلة الوصل بين الذاكرة و IR هما

السجلات MBR و MDR "عمليات ميكروية"

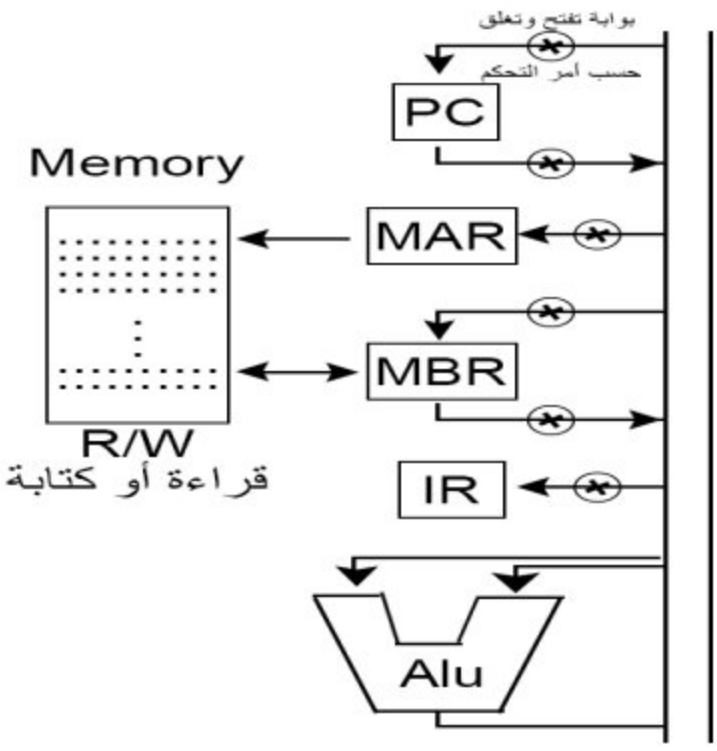
_ تعليمة ميكروية :

طور الجلب Fetch Cycle

T1 : MAR ← (PC)

T2 : MBR ← mem

T3 : IR ← MBR , PC ← PC + I



زمن النفاذ للذاكرة

الشرط : Access Time < I1 - I2

حيث : # Number of Byte of instruction : I

أنماط العنونة

عنونة فورية : ADD Ax , 05

عنونة السجل : سجل يحوي القيمة : ADD Ax , Bx

عنونة غير مباشرة : "السجل يحوي عنوان القيمة في الذاكرة :

ADD Ax , [Bx]

OP --> زمن التعليمة

شكل التعليمة : OP.Code / Addressing Information

معلومات العنونة للتعليمة

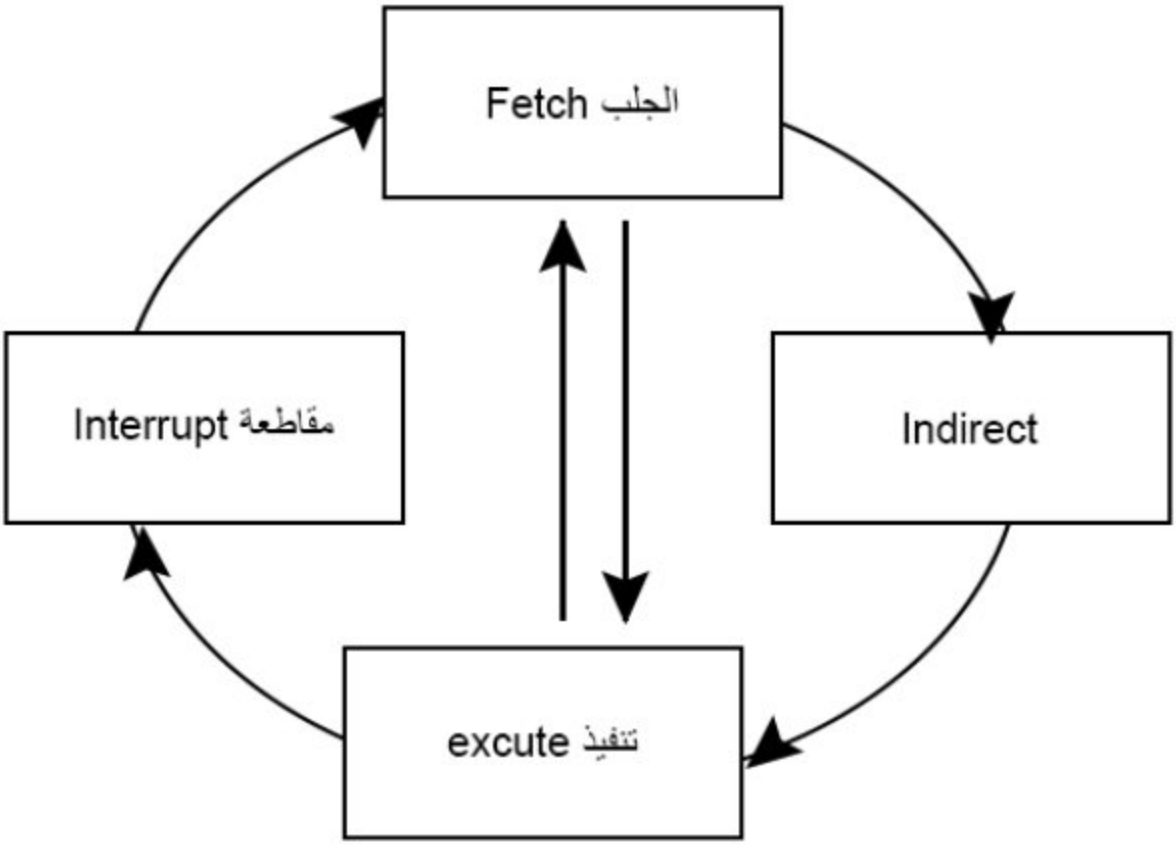
ملاحظة :

المقاطعة : عند تنفيذ مجموعة من التعليمات وعند تعليمة معينة ورد أمر طارئ للمعالج فيترك التعليمات وينفذ هذا الأمر وعند الانتهاء يعود لاستكمال تنفيذ التعليمات

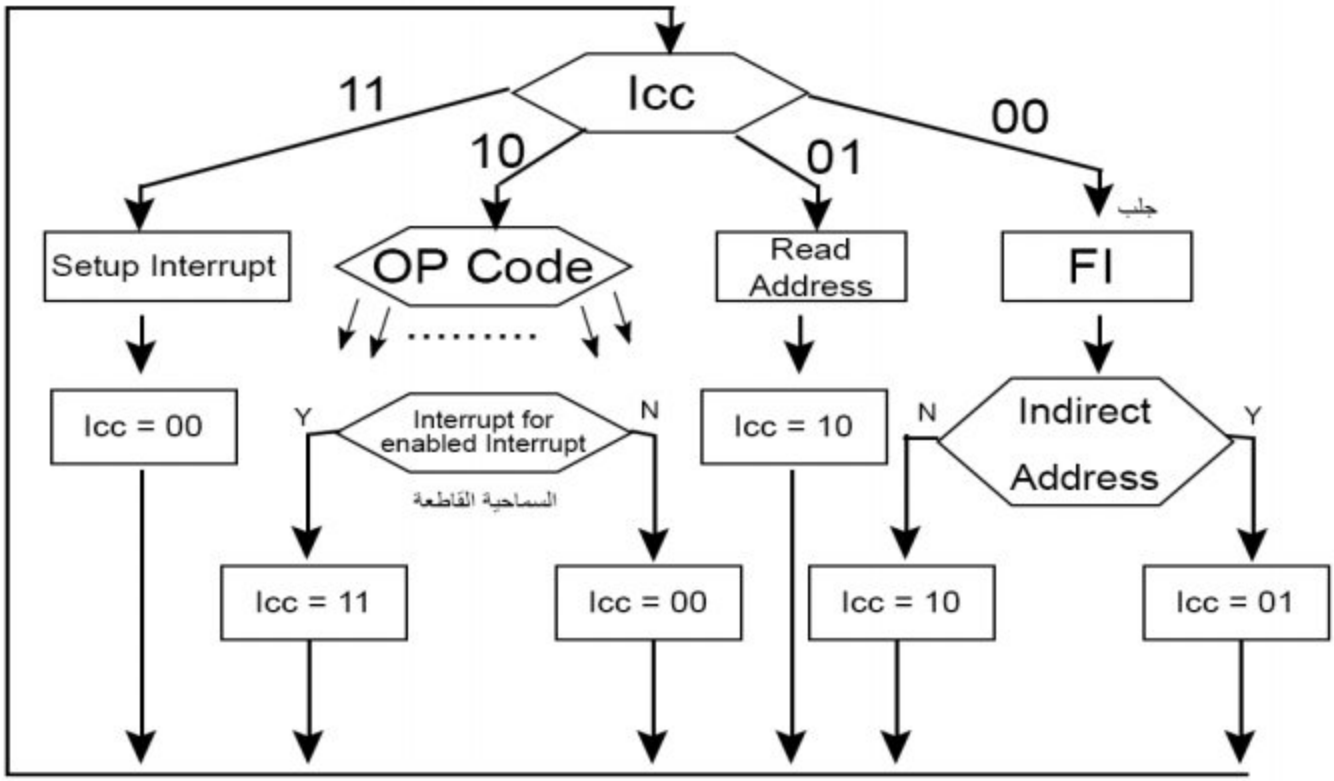
أطوار التعليمة :

1) المخطط العام :

- يبدأ بطور الجلب
- يقوم بالفحص : عنونة غير مباشرة --> Indirect --> excute
  عنونة فورية --> excute
- ممكن أن يحدث مقاطعة



) المخطط التفصيلي :



سجل (الإعداد) مؤلف من bit – 2 لمعرفة المعالج بأي طور

الطور	Q	P
Fetch	0	0
Indirect	1	0
Execute	0	1
Interrupt	1	1

المحاضرة الخامسة

العمليات المكروية

ملاحظة :
نبه الدكتور على أهمية هذه المحاضرة (من 15 إلى 20 درجة)

في هذه المحاضرة قادرين أن نصمم وحدة تحكم

كل تعليمة في المعالج تحول إلى عمليات مكروية بأزمنة معينة

الكتابة التي باللون الأزرق لا تدخل ضمن التعليمة إنما هي من مرحلة التصميم التي تأتي في آخر المحاضرة

هنا القوس يعني أنه أخذ نسخة copy

الجلب Fetch Cycle

T1 : MAR ← (PC) c0

T2 : MBR ← mem c1

T3 : PC ← (PC) + I , IR ← (MBR) c2

c3

هنا حرف I وليس رقم 1 و هو Nmber of bytes of feched Inst

ينتهي طور الجلب عندما تصبح التعليمة في IR

وهنا يوجد شرط وهي أن يكون T2 - T1 > Access time شرط ضروري ويجب كتابته بامتحان

نمط العنونة الغير مباشر Indirect Cycle

T1 : MAR ← (IR(addr)) c4

T2 : MBR ← mem c1

T3 : IR(addr) ← (MBR(addr)) c2

_ addr يعني عنوان بالذاكرة

_هذه العملية تتم بعد عملية الجلب

يمكن أن تتكرر

المقاطعة Interrupt

T1 : MBR ← (PC) c5

T2 : MAR ← save address , c6

PC ← Routine address c7

T3 : mem ← (MBR) c8

تزيد فعالية الكود البرمجي عند استخدام Interrupt

ADD R1,X

T1 : MAR ← (IR(addr)) c4

T2 : MBR ← mem c1

T3 : R1 ← (R1) + (MBR) c9

جمع محتوى العنوان x مع R1 ثم وضع الناتج في R1

R1 سجل يحوي قيمة أما X فهو عنوان في الذاكرة

ISZ X

T1 : MAR ← (IR(addr)) c4

T2 : MBR ← mem c1

T3 : MBR ← (MBR) + 1 c10

If (MBR) == 0 then (PC ← (PC) + I) c11

زيادة واحد على محتوى الذاكرة

وإذا كان الناتج 0 يعمل SKIP

X : عنوان بالذاكرة

اختصار التعليمة :

INCREMENT AND SKIP IF ZERO

BSA X حفظ السياق

T1 : MAR ← (IR(addr)), c4

MBR ← (PC) c5

T2 : PC ← (IR(addr)), c12

mem ← (MBR) c8

T3 : PC ← (PC) + I c3

احفظ عنوان التعليمة التي لا يتم تنفيذها بالعنوان X

ويبدأ التنفيذ من X + 1

BUSH AX

T1 : MBR ← (AX), c13

MAR ← save address c6

T2 : mem ← (MBR) c8

Bush يحفظ في الذاكرة

Ax عبارة عن سجل

Pop سحب من الذاكرة

POP CX

T1 : MAR ← save address c6

T2 : MBR ← mem c1

T3 : CX ← (MBR) c14

قد يأتي السؤال بالامتحان : (نكشة)

POP AX وليس POP CX فينبغي تغير السطر الأخير إلى AX

AX و CX بالنهاية هي مجرد تسمية لسجلات

يجب الانتباه إلى ترتيب النبضات T1,T2,...

التعليمتين BUSH و POP هم نفسهم في المكدس الذي أخذناه في مادة الخوارزميات

الذاكرة عبارة عن أسطر أو بايتات ، كل سطر عبارة عن بايت أو 2bit أو 4bit

التعليمة عبارة عن :

Op code	Operands معاملات

البرنامج عبارة عن أسطر ، يأخذ المعالج كل سطر وينفذه

يختلف معالج عن آخر حسب عدد ونوع التعليمات التي يستطيع تنفيذها وعدد السجلات ويكون مذكور ذلك في data sheets للمعالج

كل تعليمة لها ترميز

CPU تقسم إلى ALU و CU وبينهما سجلات

سجل ال PC يأتي بالتعليمة التالية للتنفيذ

السجل : عبارة عن دارة متكاملة

تذكرة :

يوجد في الذاكرة سجلات فورية مثل R1 ويكون عددهم محدود

ملاحظات على المخطط :

R/W إشارة تحكم هل قراءة أم كتابة

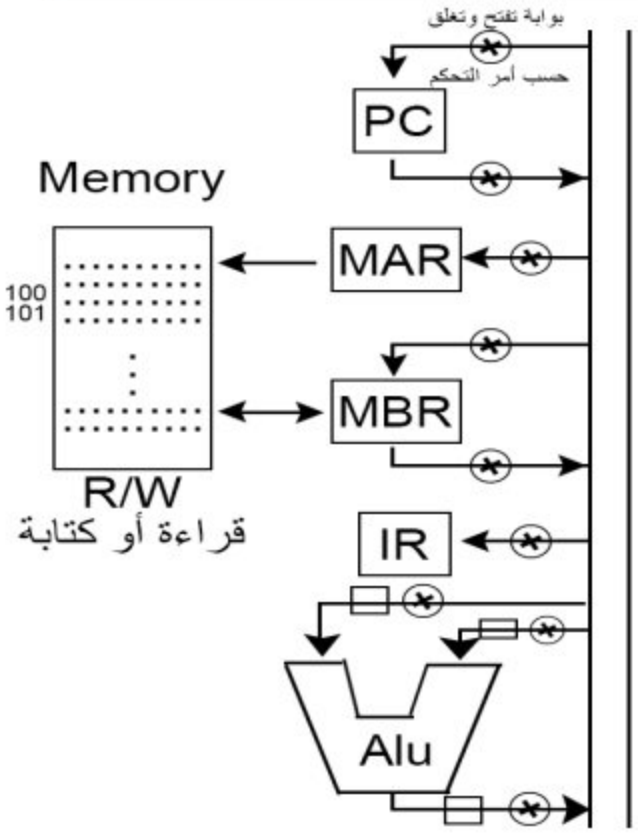
PC و IR ليس لهم صلة مع الذاكرة

لقراءة أو تخزين شيء في الذاكرة يكون عن طريق MBR حصرا

MAR و IR لا يعود إلى BUS

في ال pc عنوان فقط

المخطط بشكل عملي يحوي سجلات أخرى



لكي أصدر إشارة تحكم يلزمني :

- ما هي التعليمة
- بأي طور P و Q
- معلومات عن الزمنية (بأي نبضة) .... , T1 , T2

تذهب إشارة التحكم إلى كما أخذناها في أولى المحاضرات (... , ADD , SHIFT)

وتفيد في فتح وإغلاق الصمامات (والمرتبطة ب BUS)

ALU كي تعمل بحاجة إلى إشارات من CU

CU قد تحتاج نواتج من ALU

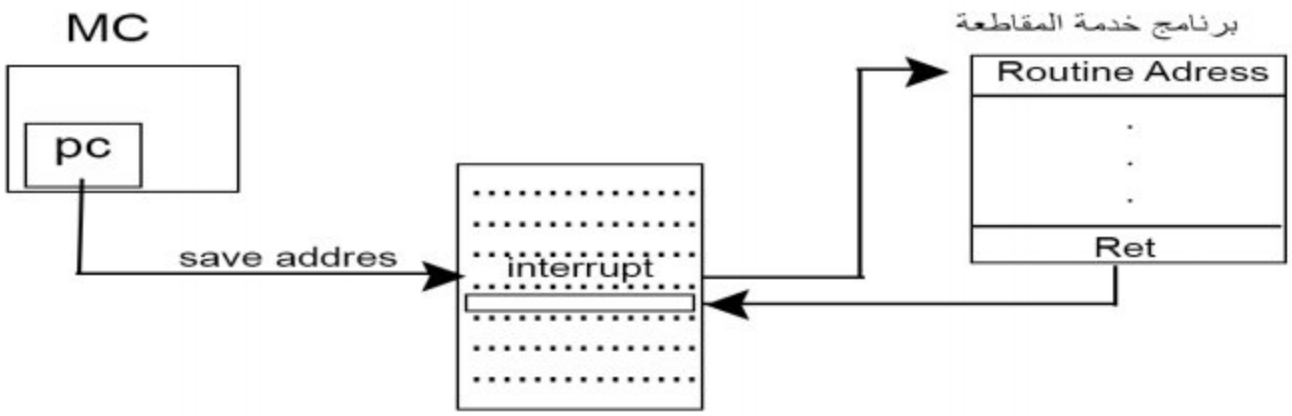
النتيجة : ALU و CU مرتبطين مع بعضهما

شرح بعض التعليمات :

- المقاطعة : Interrupt

عند تنفيذ البرنامج يكون ال pc مشير إلى عنوان ما يسمى save address

وصلنا إلى تعليمة المقاطعة عندها : لا يكمل البرنامج السطر التالي من الكود بل يذهب إلى برنامج يسمى برنامج خدمة المقاطعة أول تعليمة فيه تسمى routine address وبعد تنفيذ هذا البرنامج يكون آخر تعليمة فيه هي ret والتي تعيده إلى المكان الذي توقف عنده البرنامج وهو save address

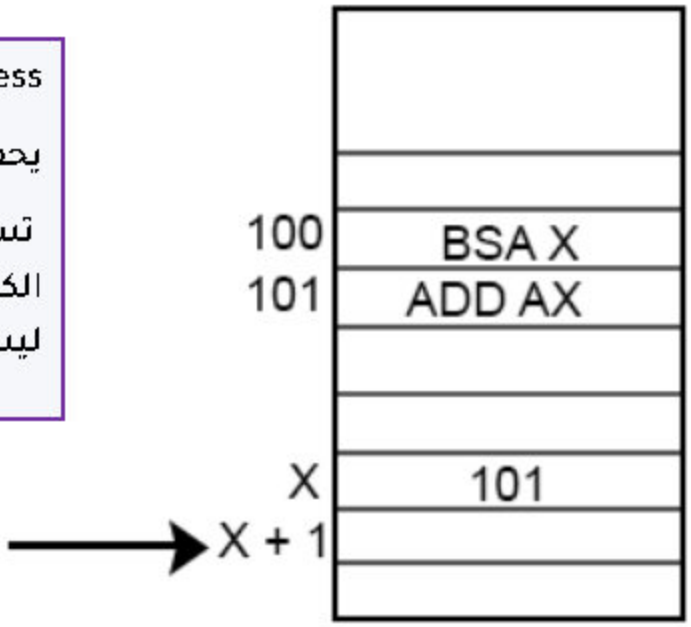


BSA X ن

Branch and Save Address

يحفظ عنوان التعليمة في X ويتابع التنفيذ من X + 1

تستخدم هذه التعليمة عند استدعاء دالة ما في الكود البرمجي فإنه ينفذ الدالة ومن ثم يعود ليستكمل تنفيذ الكود من المكان الذي توقف عنده



ISZ X ن

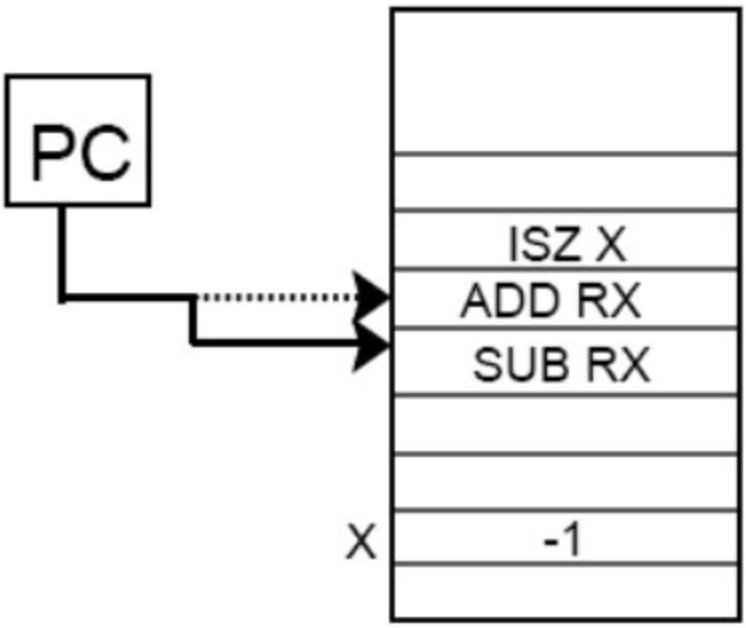
زيادة واحد على محتوى الذاكرة وإذا كان الناتج 0 يعمل SKIP

في هذه الصورة الموضحة كان pc يشير المفروض أن ينفذ تعملية ADD بعد تعليمة ISZ لكن جاءت تعليمة ISZ فأضاف 1 إلى محتوى العنوان X فوجده 1- وعند الإضافة أصبح 0

فعمل skip أو تجاوز عن تعليمة ADD إلى التعليمة التي بعدها

لو كان الناتج غير 0 كان نفذ تعليمة ADD

ملاحظة : تعليمة ADD هي مجرد مثال



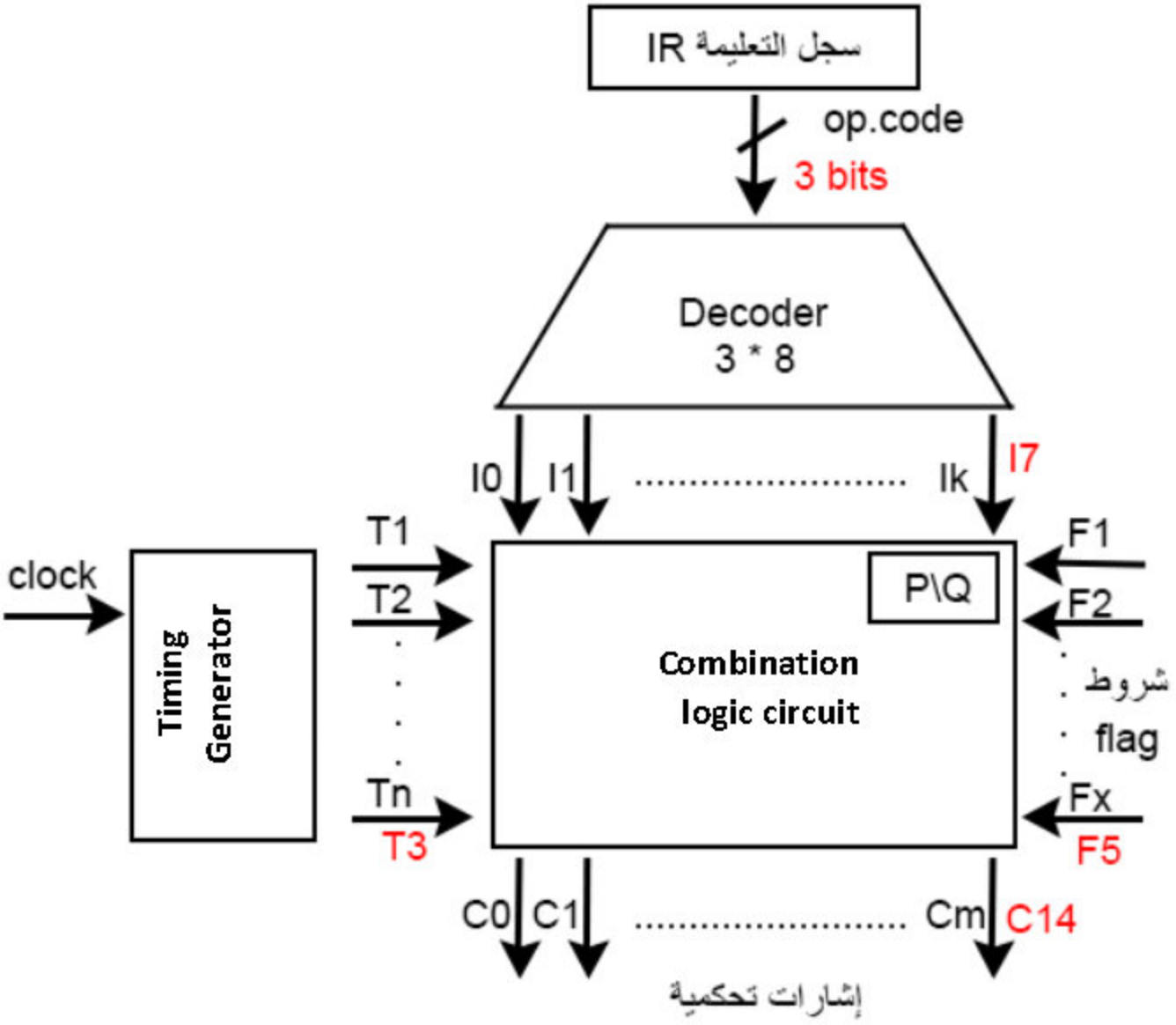
CU : تقسم إلى وحدة التحكم ذات العتاد الصلب Hardwire

والبرمجة الميكروية Micro programming

لتصميم وحدة التحكم الميكروية : لدينا ثلاث خطوات

- تحويل كل التعليمات في المعالج وكتابتها بشكل تعليمات ميكروية
- إسناد إشارات تحكمية فريدة لكل عملية والتي كتبناها بجانب التعليمات باللون الأزرق (..... c0,c1,c3) وهي فريدة أي في حال تكرارها بتعليمة أخرى تأخذ نفس الرمز
- تختلف من وحدة إلى أخرى.

مخطط البنية الصلبة (هام) : يذكر لنا عدد النبضات وعدد الشروط flag وعدد التعليمات والمطلوب الرسم مع ذكر عدد البتات التي عليها جزء كبير من العلامة



في المثال السابق أخذنا 8 تعليمات كمثال –> أي لدينا 3bits وقد يتغير بالامتحان (30 مثلا وعندها يكون لدينا 5bits)

الذي باللون الأحمر من الضروري وضعها على الرسم وعليهم العلامات أي نستبدل Tn ب T3 ونستبدل Fx ب F5 وهكذا ...

مخرج 8 * 3 decoder هو 8 أي من I0 إلى I7

عدد الشروط يعطى بنص السؤال

عدد النبضات هي 3 كما وجدنا في التعليمات السابقة

عدد إشارات التحكم ....c0,c1,c2 وجدناها 14

مثال تصميم إشارة تحكم C1 :

$$C_1 = \bar{p}\bar{Q} \cdot T_2 + \bar{p}Q \cdot T_2 + p\bar{Q} \cdot T_2 \cdot ADD + p\bar{Q}T_2 I_{sz} + p\bar{Q}T_2 . pop$$

P و Q نأخذها من الجدول التالي (P) تعني أن P في الجدول يأخذ 0 وهكذا .....

الإشارة السابقة C1 تكررت خلال التعليمات 5 مرات لذلك هي مجموع خمس حدود

الإشارة السابقة تتعلق ب P و Q والنبضة (T1,T2 ,...) والتعليمة (ADD, ISZ, POP,....)

التعليمات الموجودة في الجدول التالي والتي هي (fetch,ind,int) لا تكتب بالتصميم

يجب كتابة الجدول التالي في الامتحان :

الطور	Q	P
Fetch جلب	0	0
Ind العنونة الغير مباشرة	1	0
Exec ما عدا ذلك	0	1
Int مقاطعة	1	1

يمكن تشكيل الإشارة بالبوابات المنطقية التي أخذناها بمادة الدارات المنطقية حيث الضرب بوابة and والجمع بوابة or

المحاضرة السادسة

بنية الحاسوب

يقسم إلى:

- ALU
- CU والتي تقسم إلى:
  - HW: وحدة التحكم ذات البنية الصلبة (بوابات منطقية) والتي انتهت في المحاضرة الماضية
  - Microprogrammed: وحدة التحكم المبرمجة مايكرويا

كلا الوحدين في CU عند التصميم تتشابه في أول خطوتين

- الخطوة الأولى: نحول كل التعليمات إلى تعليمات مايكروية.
- الخطوة الثانية: إسناد رمز فريد أو خاص إلى كل تعليمة (c0,c1,.....)
- الخطوة الثالثة في التصميم هي التي تختلف : إما نذهب لتصميم دارة مكونة من بوابات منطقية وهي النوع الأول (ذات البينة الصلبة / الثابتة) أو أخزن كل التعليمات في ذاكرة تسمى ذاكرة التحكم فهي وحدة التحكم المرمجة مايكرويا

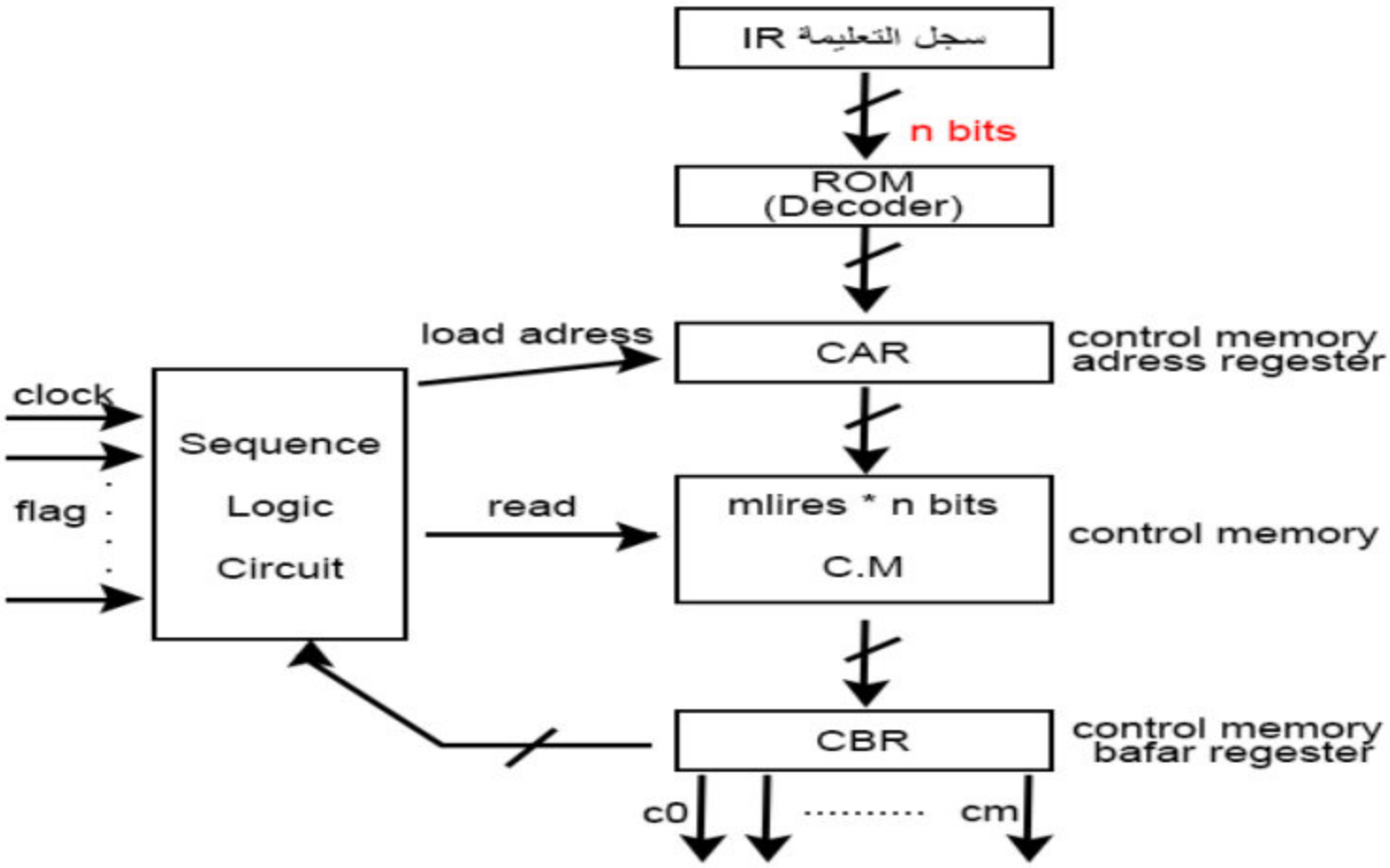
خصائص وحدة التحكم المبرمجة مايكرويا:

- قابلة للبرمجة
- قابلة للتطوير
- البرنامج يسمى fimewere

مقارنة بين وحدة التكم ذات البنية الصلبة HW وبين وحدة التحم المبرمجة مايكرويا

من حيث	HW	Microprogrammed
السرعة	أسرع من microprogrammed	أبطأ من HW
قابلية البرمجة	غير قابلة للبرمجة	قابلة للبرمجة
البنية	صلبة (ثابتة)	ليست صلبة (مبرمجة)
عدد التعليمات (التعقيد)	يمكن اختيارها في حال كان عدد التعليمات قليل (أقل من 37 تعليمة)	يمكن اختيارها في حال كان عدد التعليمات كبير جدا
التطوير	غير قابلة للتطوير	قابلة للتطوير
التكلفة	مكلف أكثر لأنه يحتاج مساحة (رقعة للبوابات)	كلفة أقل

المخطط الصندوقي لوحدة التحكم المبرمجة مايكرويا



التعليمة تكون ب IR ثم تنتقل إلى decoder من نوع ذاكرة Rom

Sequence logic circuit: قد يأتيه أمر من cbr أو من flag فينفذ load address أي حمل عنوان بالذاكرة واقفز عليه أي لا ينفذ البرنامج سطر سطر وإنما يقوم بقفز

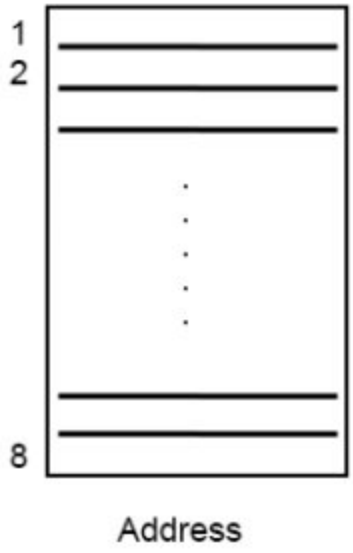
توضيح Rom (هام)

عبارة عن ذاكرة لها أسطر دخلها يكون من IR

ومن المهم معرفة كم سطر

مهمتها إعطاء عنوان السطر الأول لكل تعليمة

ففي الصورة لدينا 8 تعليمات على سبيل المثال



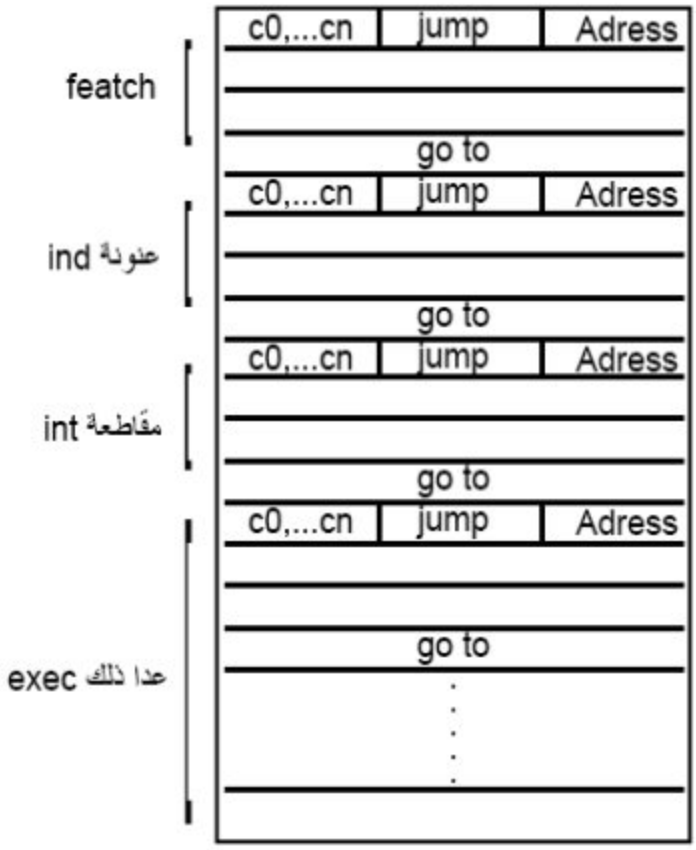
توضيح تنظيم ذاكرة التحكم C.M (هام) قد يأتي بسؤال منفصل

الذاكرة يمكن برمجتها وهي مقسمة إلى أسطر وكل سطر مكون من ثلاثة أجزاء وهذه الأجزاء تختلف إذا التعليمة أفقية أو شاقولية وهي : الجزء الأول فيه كل إشارات التحكم (c0,c1,....) - الجزء الثاني jump - الحزء الثالث يحوي Address يدل على الخطوة التالية

كل تعليمة تأخذ عددا من الأسطر حسب عدد أطوارها على سبيل المثال : تعليمة الجلب fetch لها ثلاث أطوار (T1,T2,T3) فتأخذ ثلاث أسطر في الذاكرة وتأخذ أيضا سطر إضافي هو go to يشير إلى التعليمة التالية حسب الشكل (a)

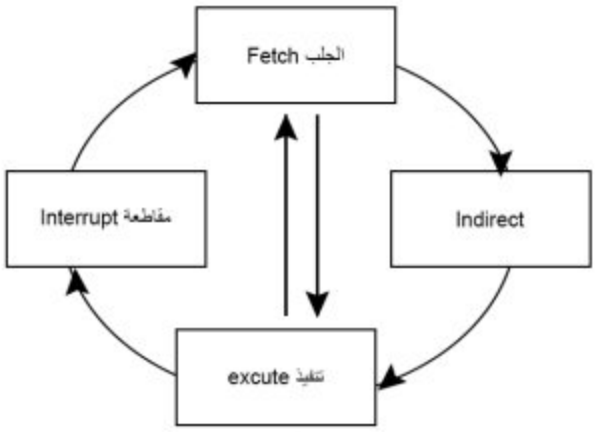
عنوان التعليمة الأولى يأتي من IR

عرض الذاكرة تختلف من ذاكرة إلى أخرى



وظيفة ذاكرة التحكم (يأتي نظري)

تخزين إشارات التحكم



الشكل (a)

مثال رقمي

ملاحظة:

قد يأتي معطيات لا نحتاجه ا فقط كي نضيع بالامتحان 🤔 (نكشة)

83 Instruction

7 flags

45 Micro operation (control signal)

أطول تعليمة تأخذ 3 نبضات

صمم وحدة تحكم بناءا على المعطيات السابقة

الحل:

المطلوب منا إيجاد طول وعرض كل من Rom و Control Memory

لدينا 83 تعليمة فالعدد كبير فحتما نصمم وحدة تحكم مبرمجة مايكرويا ولا يمكن تصميم ذات البنية الصلبة

أعطانا 83 تعليمة فنحتاج 7bits من أجل عنونتها

$$2^7 = 128$$

نتج لدينا 128 وهذا العدد أكبر من 83 ولا مشكلة ، المهم ألا يكون أصغر من 83 ولذلك ما أخذنا 6 لأن

$$2^6 = 64$$

ومنه الناتج الذي حصلنا عليه وهو 128 يكون طول الذاكرة Rom (عدد أسطرتها)

ولمعرفة طولها لدينا من المعطيات نحسب طول C.M ومنه نحسب طول ال Rom :

لدينا 83 تعليمة (instruction) وأطول تعليمة تأخذ 3 نبضات أي 3 أسطر ولدينا سطر لـ go to أي أصبح لدينا لكل تعليمة 4 أسطر

$$4 * 83 = 332$$

نضيف عليهم التعليمات (interrupt, indirect, fetch) ولكل منهم أربعة أسطر كذلك فينتج

$$332 + (3 * 4) = 344$$

لا يمكننا عنونة الذاكرة بهذا العدد حيث يجب أن يكون العدد ناتج عن العملية : 2^n

حيث n عدد صحيح ويكون n هو عرض ال Rom وهو نفسه عدد البتات اللازمة لتمثيل ال address

فيكون أقرب عدد يحقق ذلك هو 512 حيث حصلنا عليه من 2^9

وهذا العدد (512) هو طول ال control memory

ومنه نتج عندنا عرض ال Rom وهو 9

حصلنا على بعد الذاكرة Rom وهو 9 * 128

يتبقى علينا حساب عرض ال control memory

حصنا على عرض c.m من شكل التعليمة الأفقية والتي هي كما يلي:

Control signals	jump	c.m address
45	3	9

لدينا من المعطيات 45 إشارة تحكم وضعناهم تحت control signals

لدينا من المعطيات 7flags وترميز العدد 7 هو 3bits لذلك وضعنا 3 تحت jump

وبقي لدينا جزء address وقد حسبناه ووجدناه 9

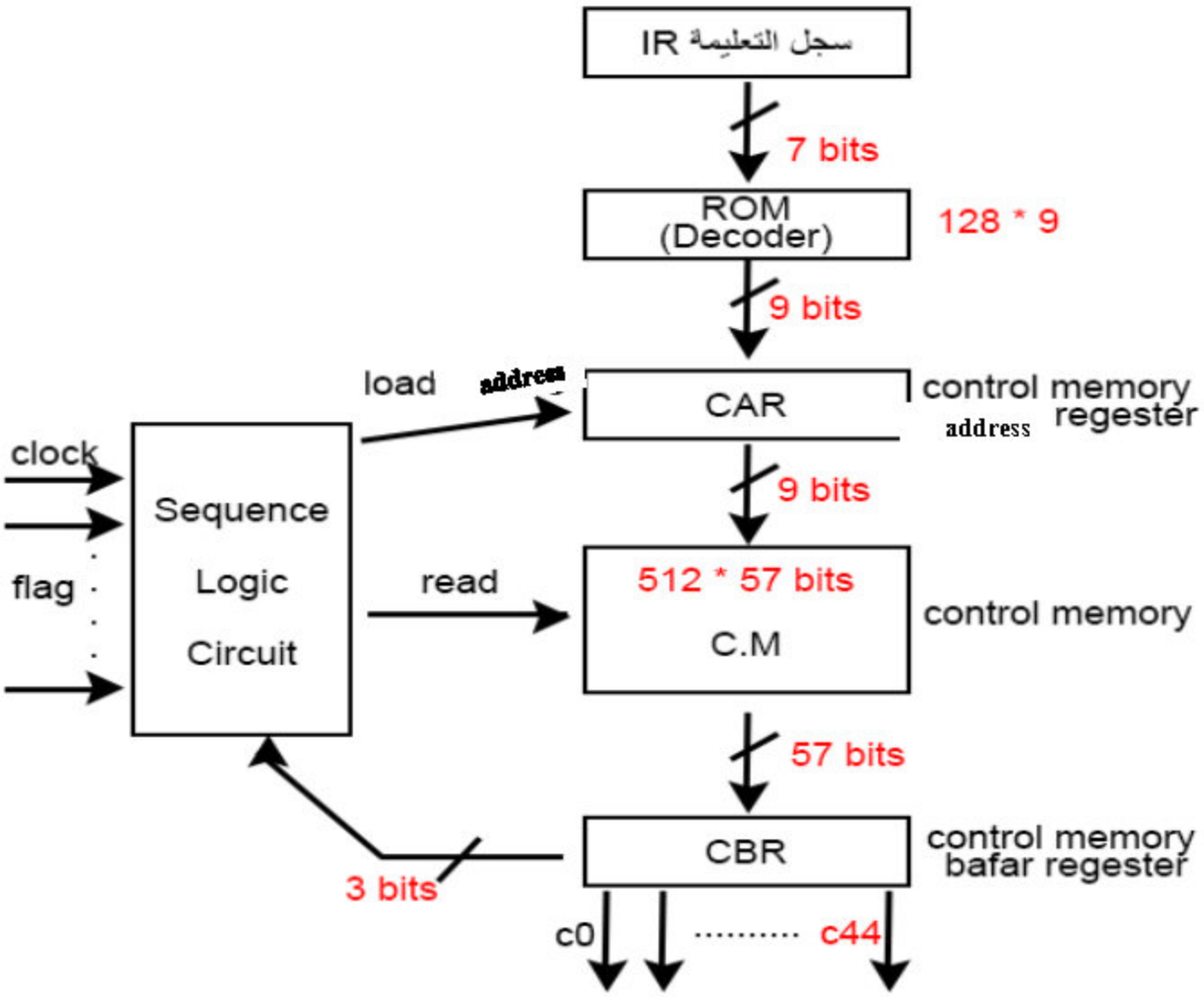
المجموع هو عرض control memory وهو 57

ملاحظة:

بالامتحان حتما تكون التعليمة أفقية لكن قد يرد السؤال الفرق بين شكل كل تعليمة

ويكون مخطط وحدة التحكم المبرمجة مايكرويا

ملاحظة هامة:
(كل سطر مكتوب بالأحمر عليه علامة بالامتحان وفي حال عدم كتابته يذهب من علامة السؤال)



قد يأتي السؤال فقط تصميم ذاكرة التحكم وذاكرة Rom أو ما يسمى decoder فرسمهم والأهم العمليات بالأرقام وفي مثالنا

9 * 128 بالنسبة ل Rom وكذلك 57 * 512 بالنسبة ل C.M

شكل التعليمة (يأتي نظري)

- أفقية : تخزن كل التعليمات المايكروية
- عامودية : ترمز التعليمات decoder قبل تخزينها / ضغط التعليمات

تكون ذاكرة التحكم في التعليمة الأفقية أعرض منها إذا كانت عامودية لإن التعليمات غير مضغوطة

هناك جزئين نفسهم في كل تعليمة وهما العنوان Address و ال jump الجزء الثالث يختلف من تعليمة إلى أخرى حسب شكلها

التعليمة الأفقية :

Control signal	Jump Code n	C.M Address

التعليمة العامودية / الشاقولية :

Function code	Jump Code n	C.M Address

Function code: أي يقسم الإشارات التحكمية إلى أقسام ويرمزإلى كل قسم برمز category

- القسم الأول: حسب العمليات (ذاكرة - عملية حسابية - نقل بين سجلات - عمليات تعاقبية خاصة ....)
- القسم الثاني: هل يقرأ من ذاكرة أم من سجل أم من i/o خارجي
- القسم الثالث: خاص بـ destination بالتعليمة الشاقولية يمكن ضغط التعليمات بعدة مراحل من ال decoder مما يقلل عدد البتات اللازمة لتمثيلها وبالتالي وفرت مساحة برقة المعالج ولكن على حساب الزمن

مقارنة بين معالجات RISC و CISC

RISC : Reduce instruction set computer

CISC : Complex instruction set computer

من حيث	RISC	CISC
عدد التعليمات	قليل	كبير
وحدة التحكم	Hardwire	Microprogrammed
العمليات المعقدة	لا يقوم بعمليات معقدة	يقوم بعمليات معقدة
السرعة	أسرع	أبطأ
عدد أنماط العنونة	واحد أو اثنان	كثيرة
عدد السجلات	قليل	كثير
التكلفة	مكلف أكثر	كلفة أقل

شرح تعليمة pipline : المعالج لا ينفذ البرنامج تعليمة وينتهي منها ثم ينتقل إلى التعليمة التالية بل يقوم بعملية pipline أي مبدأ التوابع على التفرع

فمثلا إذا كان لدينا برنامج يحوي على سبيل المثال:

- تعليمة جلب fetch
- عنونة indirect
- كتابة بالذاكرة
- .....
- ..... الخ

يبدأ بالتعليمة الأولى وهي الجلب ويأتي بعاملاتها وفي نفس الوقت يكون قد بدأ بالتعليمة الثانية وبدأ بتنفيذ وجلب معاملاتها وفي الوقت نفسه يكون قد بدأ بالتعليمة الثالثة ومعاملاتها (بارامتراتها) وهكذا ...

وتختلف عدد العمليات التي يستطيع المعالج القيام بها في الوقت نفسه من معالج إلى آخر (كلما زادت زاد كفاءة المعالج)

فهو لا ينتظر لينتهي من التعليمة الأولى ليبدأ بالتعليمة التي تليها بل ينفذ من كل تعليمة ما يستطيع

ملاحظة : في بعض الأحيان لا يلجأ المعالج إلى هذه العملية وهي عندما تكون التعليمة التالية بحاجة إلى بارامترات من تعليمة سابقة

ملاحظة :

للمزيد من المعلومات عن تعليمة pipline وعن RISC و CISC يمكن العودة إلى الكتاب

انتهت مقررالدكتور سامي...

نسأل الله التوفيق لنا ولكم ....

اكتب ملاحظات: